

Employee Management System (EMS)

Submitted by

HARSHITA BAJIYA (BTBTC21135)

For the award of the degree of

(B.Tech & Computer Science)

Under the supervision of

Mentor

RAHUL SHARMA

Software Developer, Netparam Technologies Pvt. Ltd, Jaipur

External Guide

BHANU PRATAP SINGH

Project Manager, Netparam Technologies Pvt. Ltd., Jaipur



**Faculty of Mathematics and Computing
Banasthali Vidyapith**

Banasthali - 304022

Session: 2025

Table of Contents

	Page No
Certificate.....	4
Abstract.....	5
Company Profile.....	6
Acknowledgement.....	7
Abbreviation.....	8
CHAPTER 1: OBJECTIVE	9
CHAPTER 2: REQUIREMENT ANALYSIS(SRS)	10
2.1 Requirement Specification.....	11-15
2.2 H/w and S/w Requirements.....	16-17
2.3 Feasibility Study.....	18-19
2.4 Product Functions.....	20-21
2.5 Use-case Diagrams.....	22
CHAPTER 3: SYSTEM DESIGN ANALYSIS (SDS)	23
3.1 High-level Design.....	23
3.2 Class Diagrams.....	24-26
3.3 ER Diagram.....	27
3.4 Database Design.....	28
3.5 Data flow diagrams.....	29-31
3.6 Activity Diagrams.....	32
3.7 Flowcharts.....	33-34
3.8 Sequence Diagrams.....	35-37

CHAPTER 4: CODING

4.1 IDE's used for the project.....**38-40**

4.2 Coding part.....

1. Frontend.....**41-54**

2. Api Testing.....**55-59**

3. Database.....**60**

4. Backend.....**61-71**

CHAPTER 5: TESTING **72**

CHAPTER 6: USER INTERFACES **73-83**

CONCLUSION.....84

APPENDICES.....85

REFERNCES..... 86

CERTIFICATE



NetParam Technologies
Private Limited

NTPL/JAIPUR/TS/1927 Date: 14th December 2024

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Miss Harshita Bajiya**, a student of **Banasthali Vidyapith College Jaipur**, has successfully completed her Internship under **Netparam Technologies Pvt. Ltd, Jaipur** in **Java Technology** with **Grade A+**, from **01/07/2024** to **10/12/2024**.

She worked on a project entitled **Employee Management System**.

During the Internship, she has demonstrated his skills with self-motivation to learn new skills. Her performance exceeded our expectations and He was able to complete the project on time.



Dr. Manoj Sharma
(Director)
NETPARAM, JAIPUR

*Reference for Grades

Grade	E	A+	A	B	S
Meaning	Excellent	Very Good	Good	Average	Satisfactory

747, Janpath, Rani Sati Nagar, Nirman Nagar, Jaipur- 302019, Rajasthan

1. ABSTRACT

In this era of technological developments, people tend to use more sophisticated data management with good performance. Web Pages are the face of the Internet, which helps users to access the information across the Internet. Web Pages are made by developing them using various tools and technology stacks like HTML defines structure, CSS decorates the structure, JavaScript Enables us to have Dynamic Web Pages or Applications, One of the famous stacks which is put to usage for decades is the JEE of Java and Spring with Spring Boot. Therefore these latest technologies named **Spring Boot and React** are playing a very prominent role in managing the data. Spring Boot is considered as a light-weight framework and helps in retrieving and storing the data with less complexity. React JS is extremely competent and handles dependencies. Our model helps in managing employees data in a database with Spring boot as backend and react JS as frontend. Axios is used to connect frontend with backend.

Our Project (Employee Management System based on Spring Boot and React JS) is a web application which uses Spring Boot backend and React with routing for frontend. React is renowned as one of the dominating front-end development libraries supporting component-based rendering within a single web page. Axios is the link between Spring Framework and React Library which generates a robust web application. In this project, data from users is gathered on the React web page and relayed to the backend by Axios, to be then stored in a database called HeidiSQL. Consequently, such data can be controlled by operations such as insertion, updation, viewing and deletion, all through a simple homepage.

Company:

Netparam is a software design and development company that provides end-to-end development service for web and mobile development. It work on a wide range of technologies ranging from open source to proprietary and custom built solutions. Founded in 2012, With Happy clients from various diversions, cities and state of the World. It claim to have experience with all domain of IT enabled services to convert client's ideas into profit enabled business hubs. Over the past years, It has worked with some amazing government and semi-government and private institution, small businesses, non-profits, academic institutions to make their ideas on floor. With years of experience, It has perfected the process, system and technologies that enable us to create high quality, scalable web and mobile solutions. The goal of Netparam is to always have clients end products reflect our deep engineering skills and elegant progressive design aesthetic. Netparam fundamentally believe in developing long-term strategic partnerships with our clients by exceeding expectations and a great level of transparency.

Projects:

Green Bharat: A social initiative that allows users to plant and track trees from home, designed for those without land or time.

Skill Bharat: Enhances youth skills through placement support, internships, training, and e-learning, facilitated by industry experts.

Healthy Bharat: Promotes health through the Yoga Power app, providing access to verified yoga methods to improve energy, stress relief, and overall well-being.

Social Bharat: Enables community empowerment because members can share activities, job opportunities, and other community-related info.

IMS- Institute Management System: All-purpose institute management system for managing different institute activities such as admissions, attendance, examinations, etc.

OEM - Exam Management System: An online tool for monitoring exam centers and exam conduction to ensure efficient management and oversight.

ACKNOWLEDGEMENT

I thank all the people who have in different ways assisted with the successful completion of this project and wish to express our gratitude.

I would like to thank our project mentor **Mr. Rahul Sharma** whose inspiration, suggestion, and invaluable guidance really helped us in developing the present software.

It has been possible owing to all the good and useful suggestions and cooperation extended by all my friends.

I am also grateful, in general, to my most respected parents who have shown me moral support and constant encouragement which has made it possible for us to attain this usage at this stage of academic achievement in our life.

HARSHITA BAJIYA

ABBREVIATIONS:

1. API - Application Programming Interface
2. CSS - Cascading Style Sheets
3. HTML - Hypertext Markup Language
4. HTTPS - Hypertext Transfer Protocol Secure
5. H/W- Hardware
6. JS- JavaScript
7. JSON - JavaScript Object Notation
8. OS - Operating System
9. S/W – Software
10. UI - User Interface
11. URL - Uniform Resource Locator
12. UX - User Experience

Chapter-01

2.OBJECTIVE

In today's fast-running world at work, well-working management of employee data at the organizational level is exactly what organizational success may necessitate. A central channel, through which EMS will successfully manage attendance and worktime, along with payroll about their employees, easily and accurately may come in very handy towards such a need.

Manual tracking and record-keeping, as the business grows, will begin to yield to errors, inefficiency, and increased administrative burden. EMS eliminates all these problems because it is the system that automates key activities, such as calculating the total number of working hours every month, employee records maintenance, and proper salary calculations based on the effective rates every month. It is for organizations who wish to run their activities without errors and more towards strategic growth, rather than getting bogged down by the administrative burdens.

Chapter-02

REQUIREMENT SPECIFICATION

1. Introduction

1.1 Objective

This document is to specify the requirements of the Employee Management System. EMS is a system in which all employee details and user interactions are handled with different types of roles, such as Admin and Employee.

1.2 Scope

EMS includes:

- Admin Role: All module access, such as Home, Dashboard, Profile, Login, and Signup.
- User Role: Don't have access to Dashboard and Profile only have access to home, about us, contact us, feedback form.
- Authentic login and sign-up.
- Feedback mechanism to engage users.

1.3 Target Audience and Reading Recommendations

- Project Developers: This document is for implementation.
- Testers: The functional and non-functional requirement can be referred while testing the criteria.
- Clients and Stakeholders: Known the system capability.

1.a. Requirement Analysis

1.a.1.Functional Requirements

Admin Role:

- Dashboard management.
- User account management (CRUD).
- Login and signup with security.
- All module access (Home, About Us, Contact Us, Feedback Form, Profile).

User Role:

- Access to Home, About Us, Contact Us, Feedback Form, and Profile.
- Submission of feedback via form.

1.a.2.Non-Functional Requirements

- **Performance:**

The EMS should allow efficient operation, taking into consideration server response time, user internet speed, or the like. This involves fast and responsive actions toward login record management, and report generation. Proper registration of users is thus a must to ensure no unauthorized access to the system and maintain security and efficiency as well.

- **Security:**

The EMS will utilize a secure database featuring strong encryption and access controls to safeguard sensitive information from unauthorized access. The system will guarantee that only administrators can manage critical functions, including user account management and application oversight. This centralized control is essential for maintaining data integrity and preventing misuse.

- **Usability:**

- Simple and intuitive interface for users and admins.
- Mobile-responsive design.

- **Safety Requirements**

To protect the database, the EMS needs to put in place strategies to avoid crashes and data loss from problems such as viruses or operating system failures.

- **Software Quality Attributes**

- **Availability:** The EMS requires a stable internet connection for access and mandates active user accounts, aiming for minimal downtime to ensure users can consistently access the system when needed.
- **Reliability:** The system is designed to avoid crashes and includes proper exception handling to maintain stability and data integrity. This ensures that operations like marking attendance and calculating salaries are executed smoothly.
- **Portability:** The EMS is designed to be compatible with Android devices, allowing users to access the system from various mobile platforms, which enhances accessibility and user convenience.
- **Maintainability:** The system allows for easy integration of new modules and updates, supported by comprehensive documentation. This ensures that the EMS can adapt to changing needs and incorporate new features over time.
- **Reusability:** The EMS is intended for continuous use with no time restrictions, allowing users to access the system whenever needed. This feature supports ongoing user engagement and ensures that the system remains a valuable tool.
- **Robustness:** The system is built to handle data effectively and manage user registration and privileges efficiently. This robustness ensures that the EMS remains stable and reliable, even under varying conditions and usage scenarios

1.a.3.Requirement Specification

3.1 Dashboard

- Only admin users can access the page.
- **Features:**
 - Summarizes system activities
 - Present user activity, feedback statistics, and attendance summaries.
- **Toolboxes for:**
 - Adding new employee
 - Displaying all record of employees
 - Marking attendance.
 - Managing company statistics and rate.
 - Salary calculation
 - Checking feed backs and messages

3.2 Home

- This home page is accessible to admin users and general users.
- **Features:**
 - Present general information about the Employee Management System
 - It is a log-in page for authenticated users

3.3 About Us

- This page is accessible to both admin and general users
- **Features:** Static page that contains information about the organization.

3.4 Contact Us

- **Access:** Admin and general users can access.
- **Features:** Query or message submission form.
- Fields include name, email, subject, and message.
- Submissions are sent to the system for further action.

3.5 Feedback Form

- **Access:**
 - General users can submit feedback.
 - Admin users can view and analyze feedback.
- **Features:**
 - Users can share their experience or suggestions.
 - Admin dashboard contains tools for analyzing feedback trends

3.6 Profile

- **Access:** Accessible to both admin and general users.
- **Features:**
 - Users and admins can view and edit personal details.
 - Profile section includes fields like name, email, and password.

3.7 Login/Signup

- **Access:** Secure access for all users.
- **Features:**
 - There are separate login and signup forms for the admin.
 - It applies role-based access control
 - Admins can access the dashboard and manage system features
 - Only the restricted pages "Home," "About Us," "Contact Us," "Feedback Form," and "Profile" can be accessed by the users

2. H/W AND S/W REQUIREMENTS

2.1. H/W REQUIREMENTS

Hardware Requirements Developer Side:

➤ **Computer Hardware:**

- **CPU:** A multi-core processor (quad-core or more) for smooth development experience.
- **RAM:** 8 GB or more for efficient development and testing.
- **Storage:** Sufficient free storage for the development environment, code, and project assets.
- **Operating System:** Windows, macOS, or Linux, based on the developer's choice and compatibility with the tools used.
-

Hardware Requirements User Side :

- **CPU:** A latest processor that has to be dual-core or so that web application runs smoothly.
- **RAM:** 4 GB or higher to allow multiple applications at one time and still work at a reasonable response time accessing the system
- **Storage:** Sufficient free storage for installing browsers and storing any downloaded data or reports.
- **Internet Connection:** A stable internet connection to ensure reliable access to the web-based EMS application.

2.2. SOFTWARE REQUIREMENTS

Developer Side: Software Requirements

- **Spring Boot:** A robust framework utilized for backend development, which offers the tools and ecosystem needed for web applications
- **Eclipse IDE:** · An Integrated Development Environment (Eclipse IDE) is an IDE tool for writing and managing Spring Boot applications
- **React:** Used for the front-end User Interface of the application
- **Visual Studio Code (VS Code):** A code editing tool used for frontend development that provides quite a number of plugins and tools tailored for React development.
- **Postman:** A RESTful API testing and validation tool to ensure that back end services are functioning correctly.
- **HeidiSQL:** Database management tool for used for managing MySQL databases and an easy way to download, etc. handling of application data.
- **Git:** A version control system used to keep track of changes in the codebase and facilitate collaborative development

User Side: Software Requirements

- **Web Browser:** Latest version Chrome, Firefox, Safari, or Edge
Edge, in order to access the EMS application.
- **Operating System:** Any OS compatible with the selected web browser (Windows, macOS, Linux, Android, or iOS).
- **Internet Connection:** A decent and stable internet connection for seamless access to the web application.

3.FEASIBILITY STUDY

Economic Feasibility

The EMS project will be economically feasible with very low initial costs. All that developers need is a system meeting the hardware requirements and some basic development tools. Users will mainly pay for an Internet connection. The modest investment in the system is planned to be compensated by tangible operational efficiencies, e.g., by simplifying management of personnel and validating payrolling.

Technical Feasibility

For Developers:

- **Operating System:** EMS can be developed on Windows, macOS, or Linux, so flexibility would depend on the developer's choice
- **Development Platform:** Eclipse IDE will be used for the backend using the Spring Boot framework and Visual Studio Code (VS Code) will be used for frontend using React.
- **Frontend:** React will be used for building dynamic and responsive UI.
- **Backend:** Spring Boot will be used for implementing business logic and server-side operation, thus providing a robust backend
- **Database Management:** HeidiSQL will be used for managing the MySQL database, thus providing efficient data operations.
- **API Testing:** For testing and validation of RESTful APIs, Postman will be used, thus ensuring reliable backend services.
- **Version Control:** Git will be used to manage code changes and collaboration. CI/CD tools may be integrated for automated testing and deployment.

For Users:

- **Device:** Users can access the EMS via mobile phones or computers that meet the recommended specifications.
- **Internet Connection:** A stable Internet connection is essential for accessing the EMS.

Supported Browsers: The application will be optimized for major web browsers, including Google Chrome, Mozilla Firefox, and Microsoft Edge

Behavioral Feasibility

The EMS will be user-friendly and has no special technical knowledge that must be used. The application provides an interface that is intuitive with easy navigation and displays helpful error messages. Training materials like user manuals and online help resources will also be provided. Usability testing will ascertain that the system meets user needs and provides a positive experience.

Legal Feasibility

The EMS will observe relevant data protection regulations like GDPR or CCPA to handle the management of personal and sensitive information properly. All third-party tools and libraries will be evaluated with regards to licensing compliance as well.

4.PRODUCT FUNCTION

The Employee Management System provides an all-inclusive range of functionalities that are optimized for organizational processes and user experience. These include:

1. Dashboard (Admin Only)

- Central view of system usage, user activity, and feedback statistics
- Tools for managing employee records, tracking attendance, and calculating salaries

2. User Management

- Create, read, update, and delete employee records
- Display a detailed list of employees or retrieve specific employee data by ID.

3. Attendance Management

- Mark attendance with options for full day, half day, or custom hours worked.
- Automatically calculate attendance summaries and maintain historical logs.

4.Salary Management

- Compute salaries based on attendance and applicable monthly rates.
- Generate detailed salary reports for individual employees or the organization.

5.Feedback Management

- Users can submit feedback to improve organizational efficiency.
- Admins can review and analyze feedback for actionable insights.

6.Static Pages

- Home: Overview of the system's capabilities and benefits.
- About Us: Detailed description of the organization and its mission.
- Contact Us: Query form for seamless communication with administrators.

7. Profile Management

- Users and admins can view and edit their profile details, ensuring accuracy and personalization.

8. Authentication and Authorization

- Secure login and registration for both users and admins.
- Role-based access control, limiting sensitive operations to authorized personnel.

9. Company Statistics (Admin Only)

- Display organizational metrics such as employee count, attendance trends, and performance indicators.

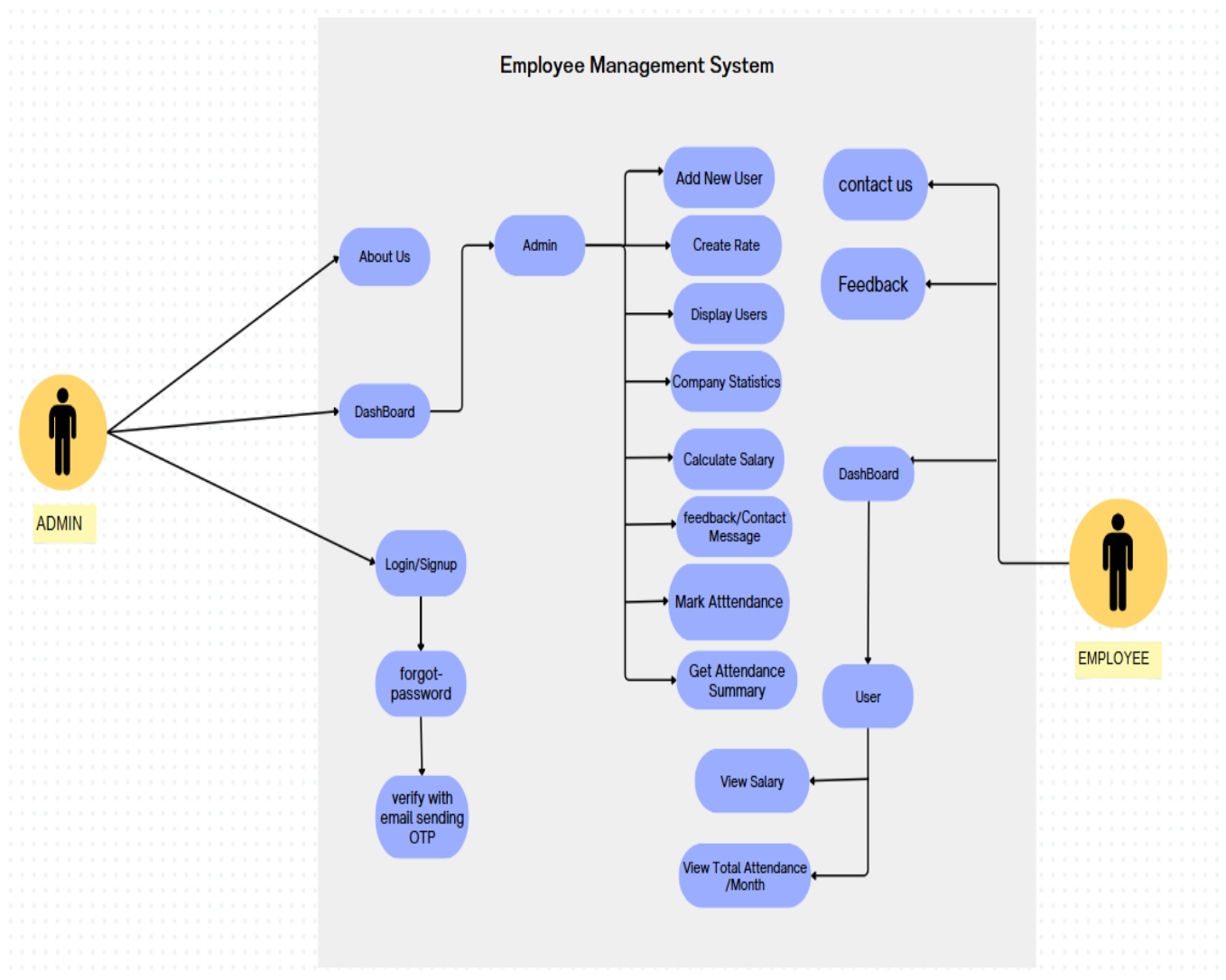
10. Message Management

- Admins can view and respond to user-submitted messages for enhanced communication.

11. Footer Section

- Contact Information: Displays an email address and phone number for user queries.
- Email: harshitabajiya2838@gmail.com
- Phone: +91-8302001419
- Social Media Integration: Links to popular platforms, such as Facebook, GitHub, Twitter, and LinkedIn, to enhance connectivity.
- Copyright Information: Displays a copyright notice, reinforcing brand identity

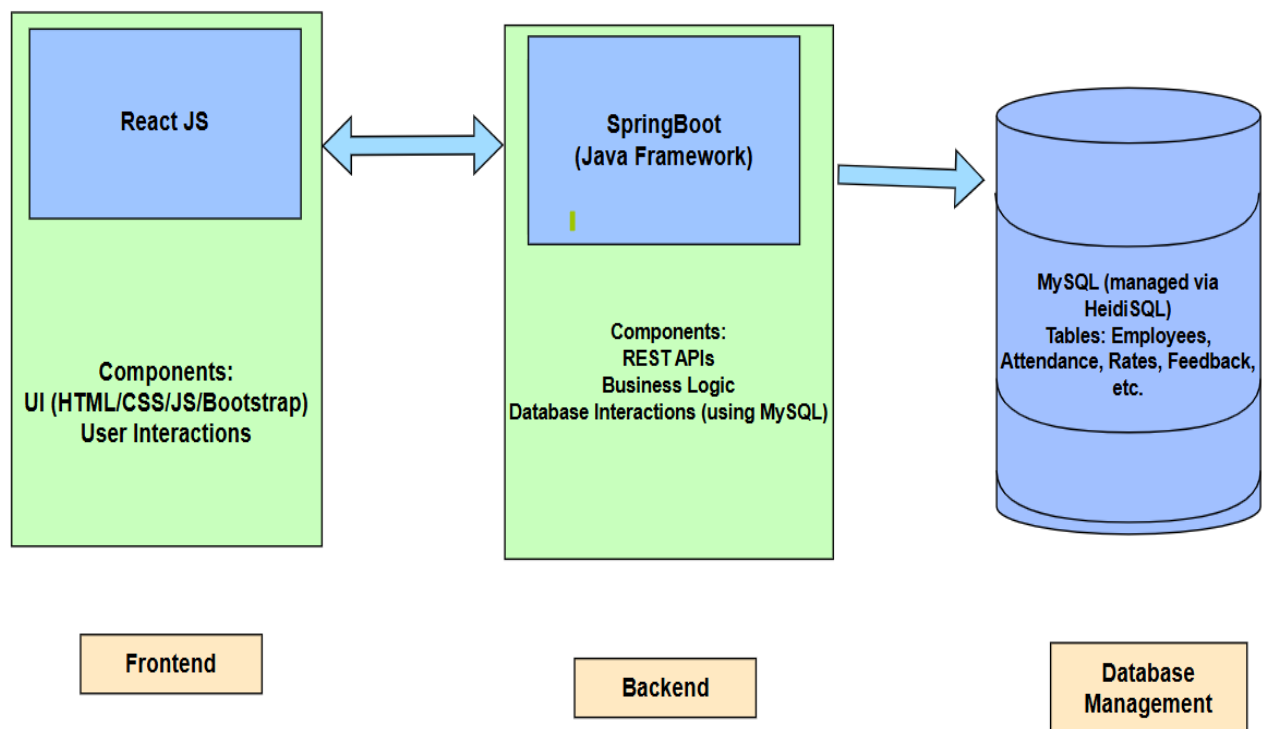
5. USE CASE



Chapter-03

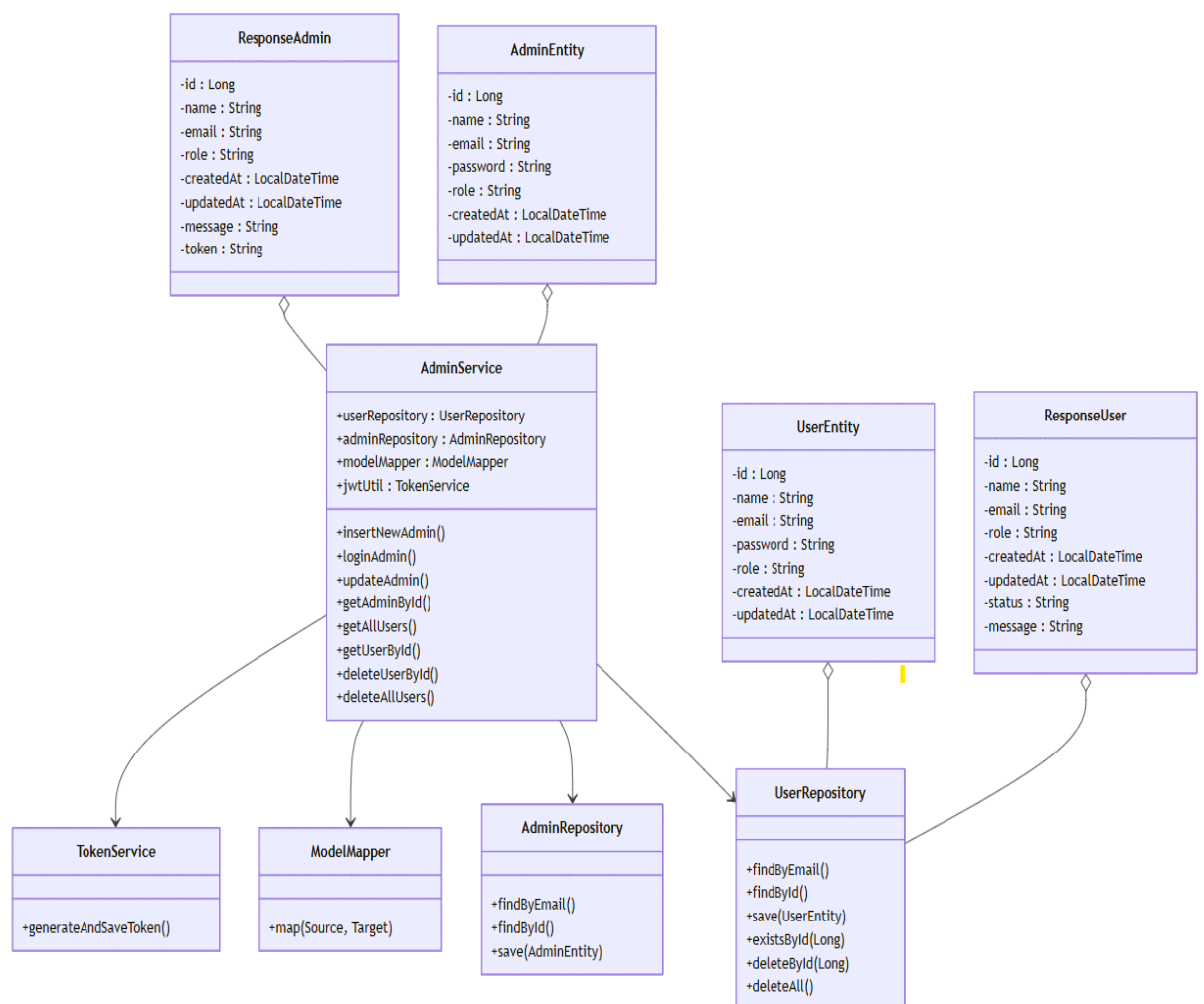
System Design (SDS)

1. High-level Design

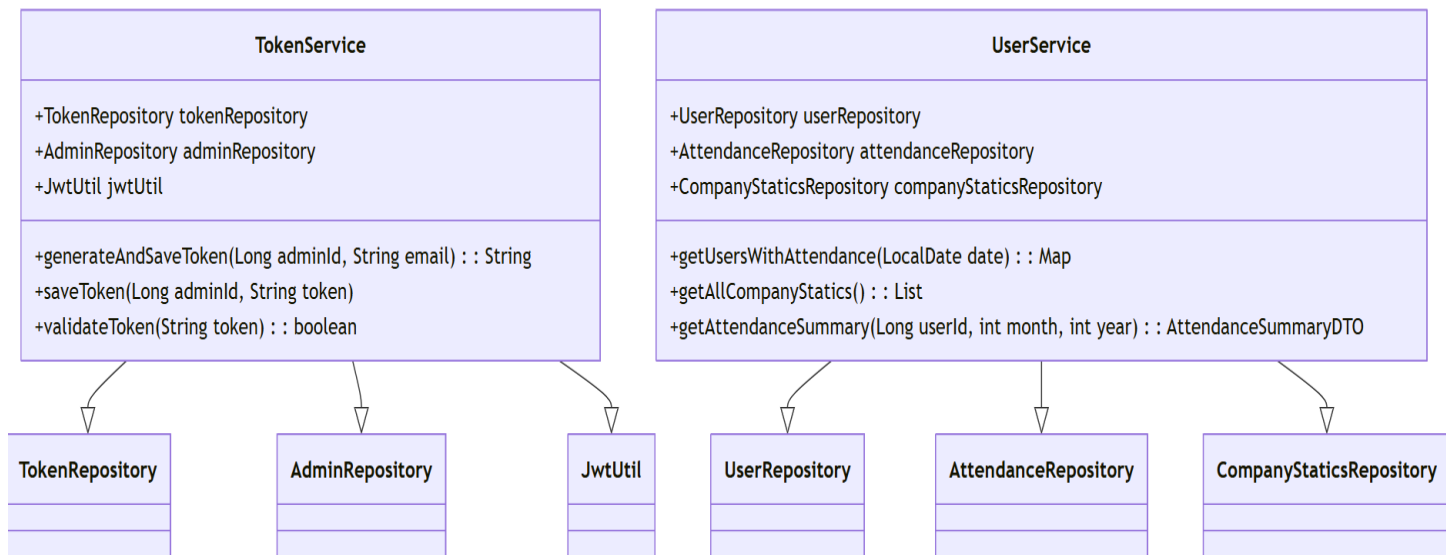


2. Class Diagrams

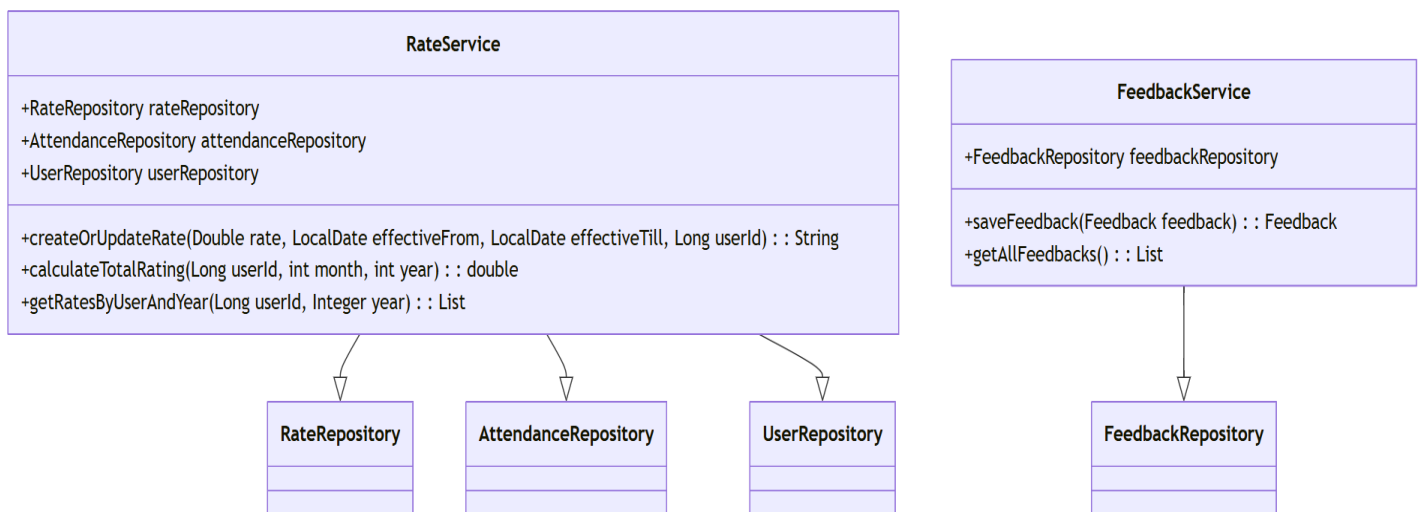
1. AdminService Class Diagram



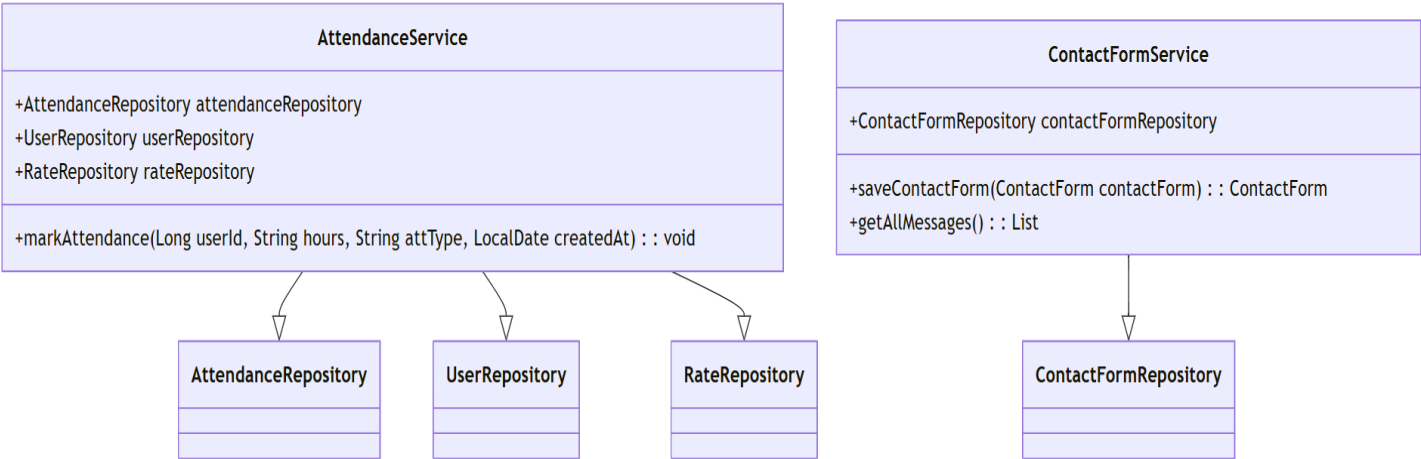
2.UserService and TokenService Class Diagram



3.RateService and FeedbackService Class Diagram

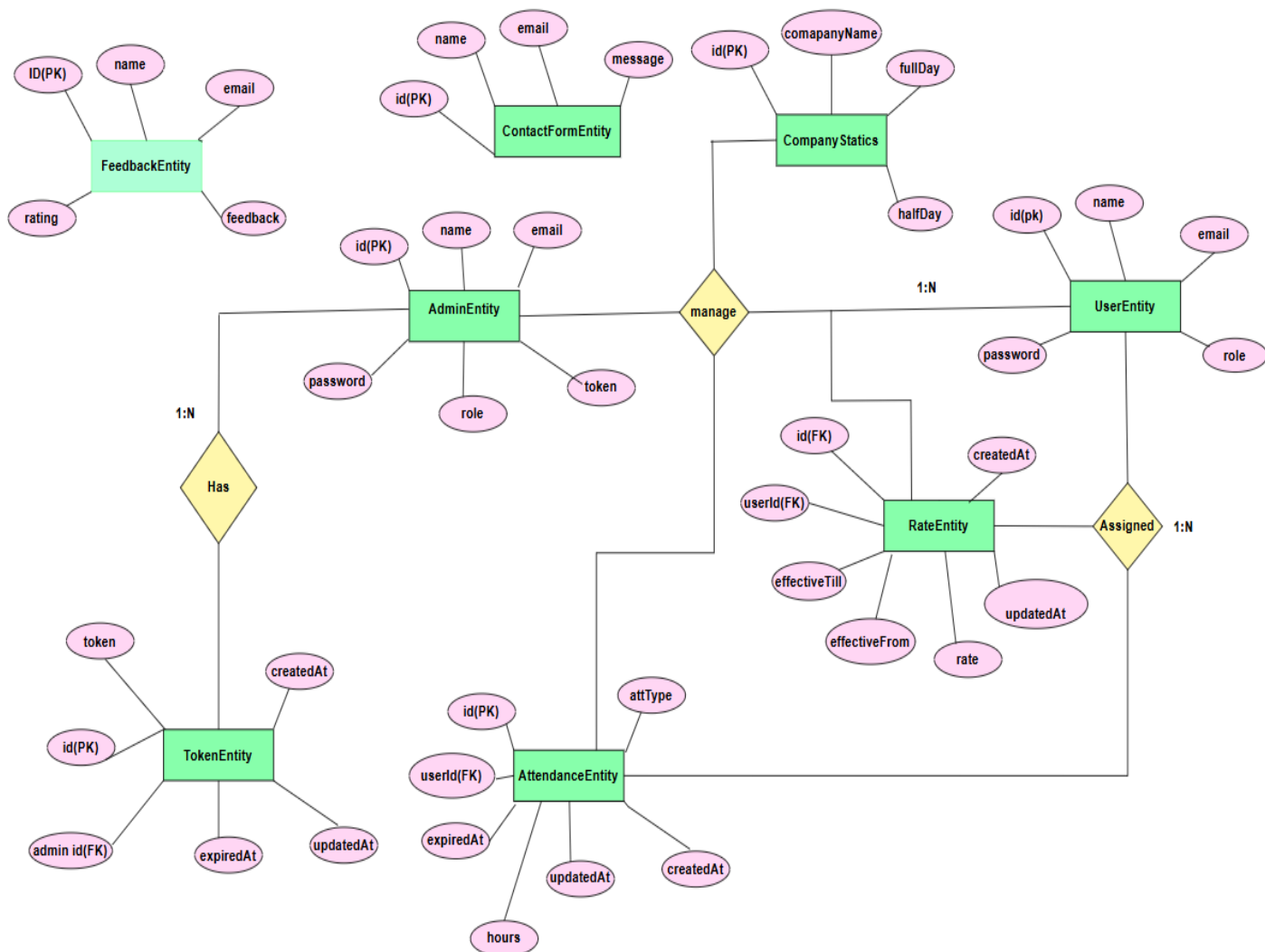


4.AttendanceService and ContactFormService Class Diagram



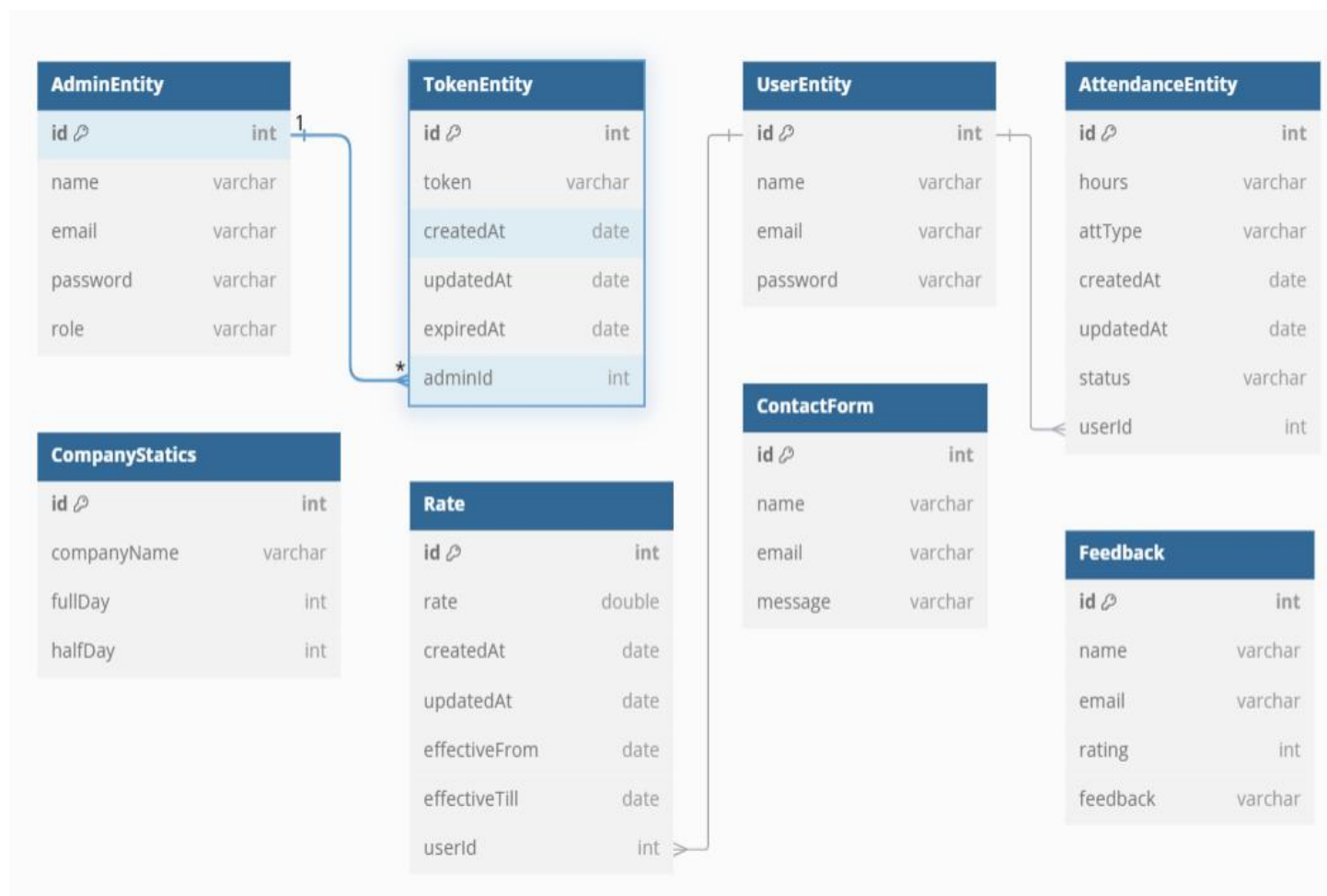
2.ER Diagram

ER diagram the database design for the employee management system project. The ER diagram is like a picture that contains how all the entities are related to each other. The most important part of the ER diagram is Relationships, Attributes, and Entities.



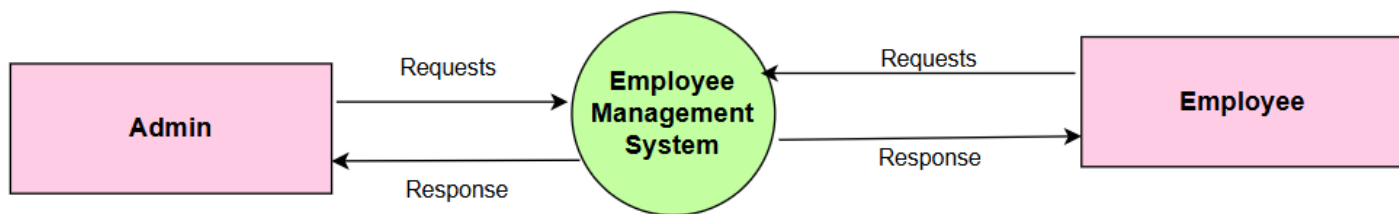
3.Database Design

The below diagram shows the database design for the employee management system. This database design shows all the system data, and the output for the user ate stored in the database



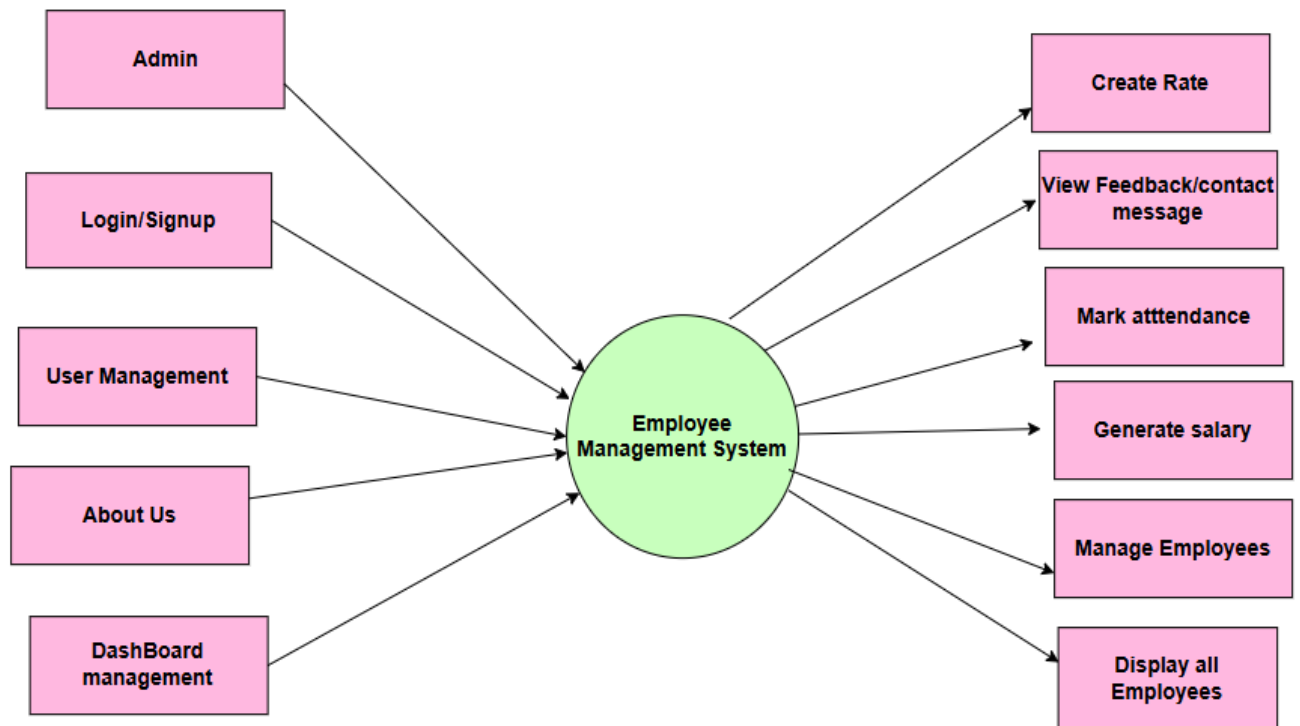
4.Data flow diagrams

1. Level- 0- DFD



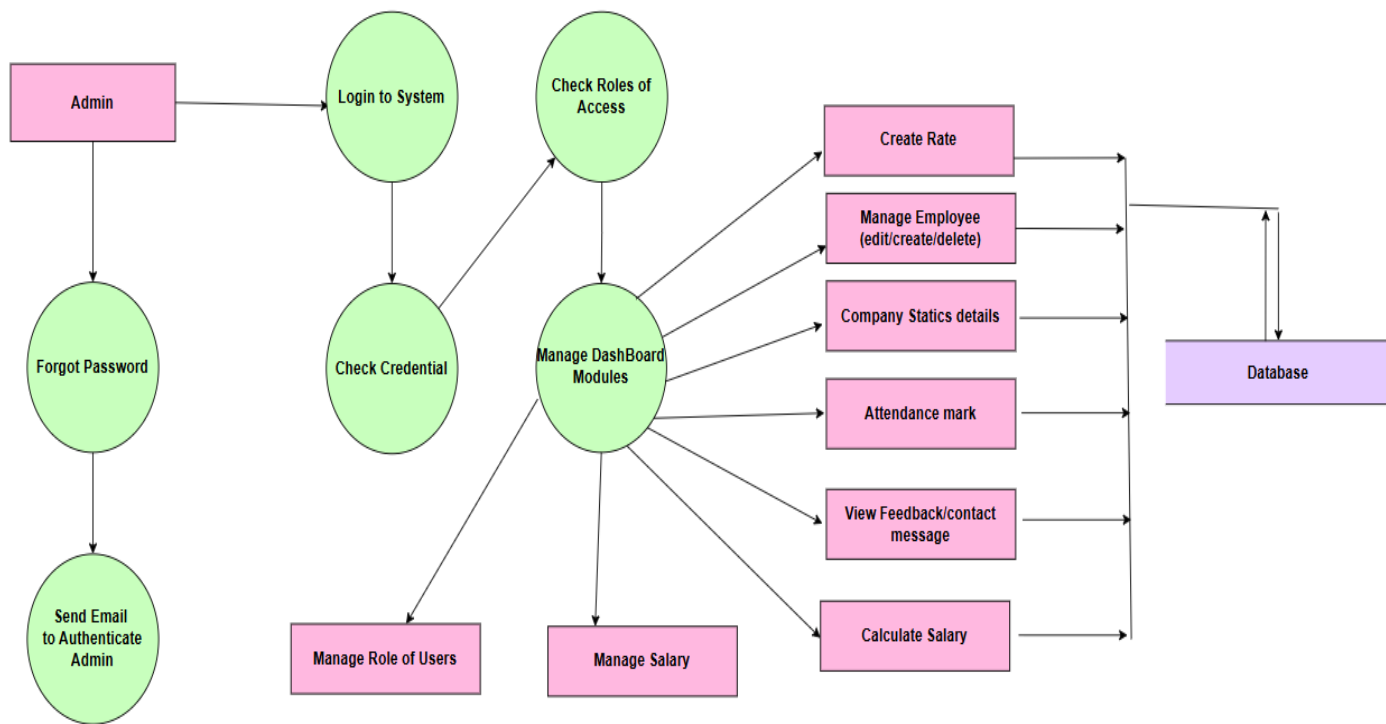
0-level DFD

2. Level- 1- DFD



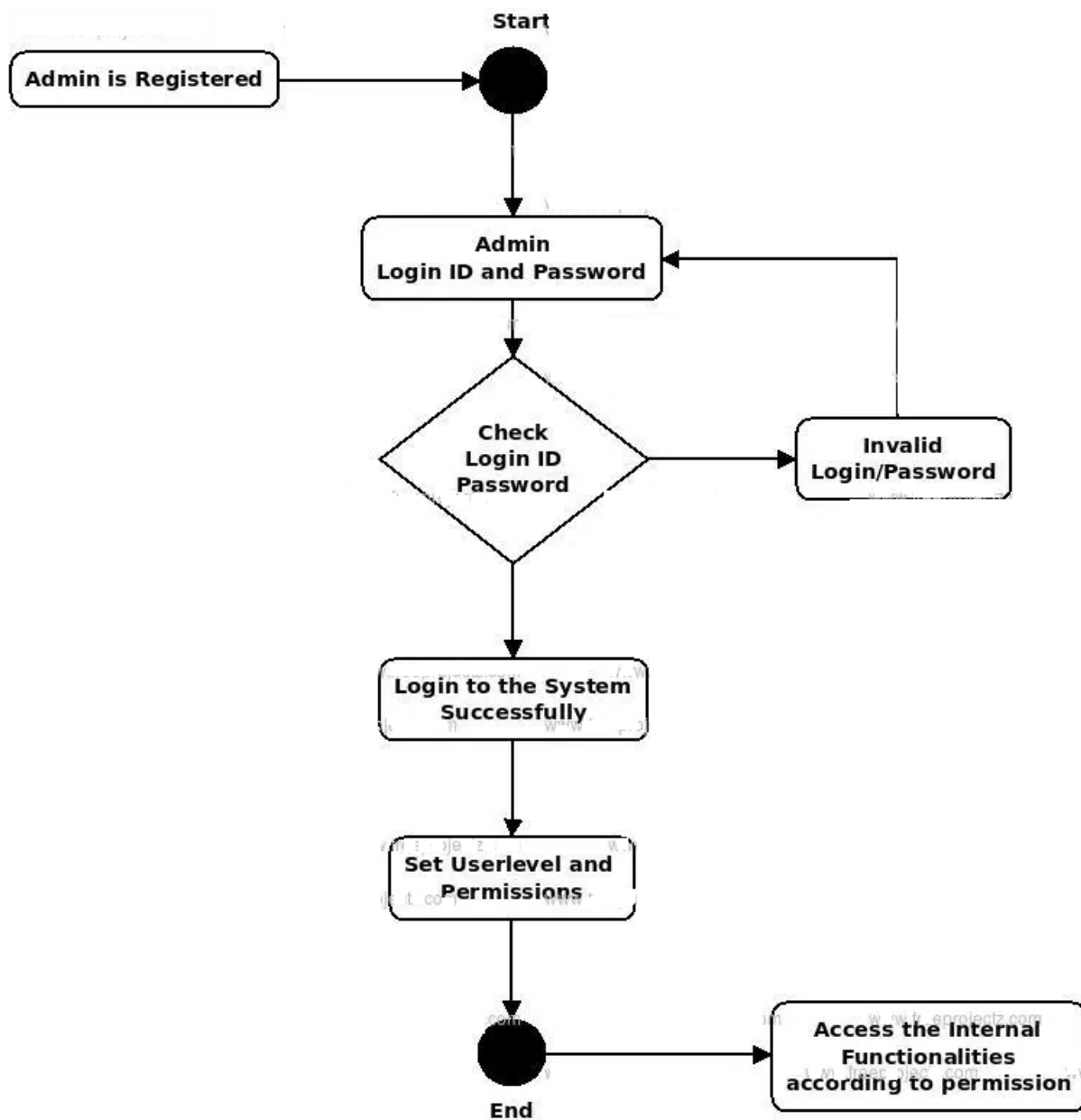
Level-1-DFD

3. Level- 2- DFD

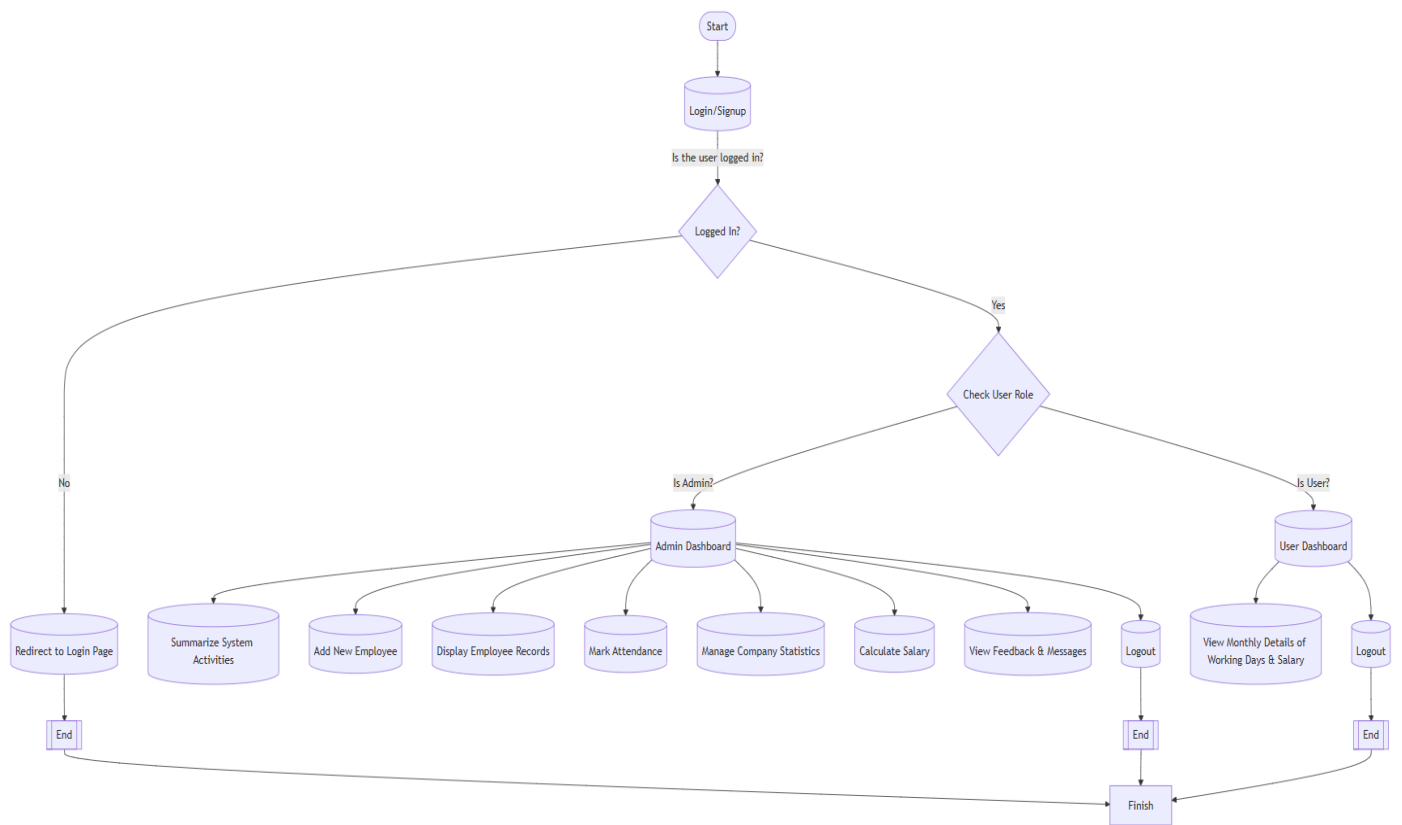


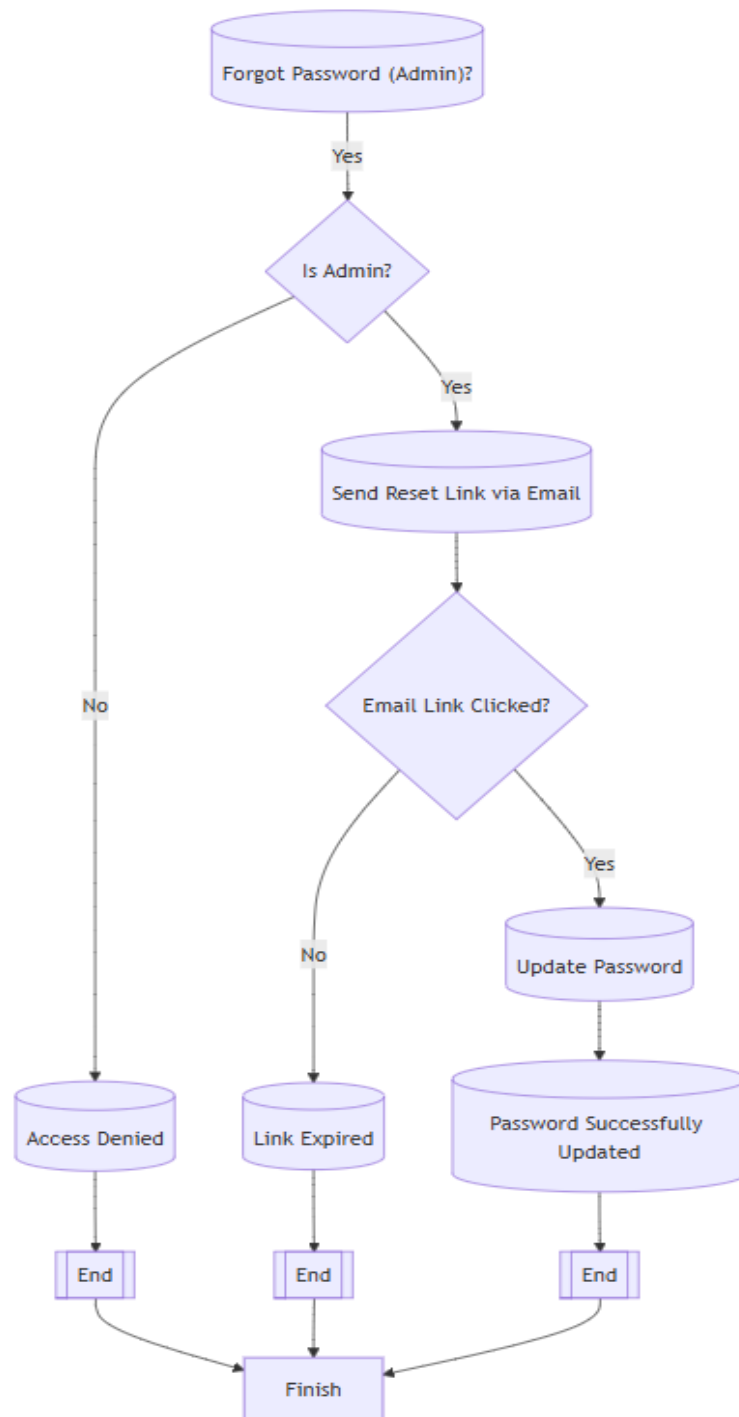
Level-2-DFD

4.Activity Diagram

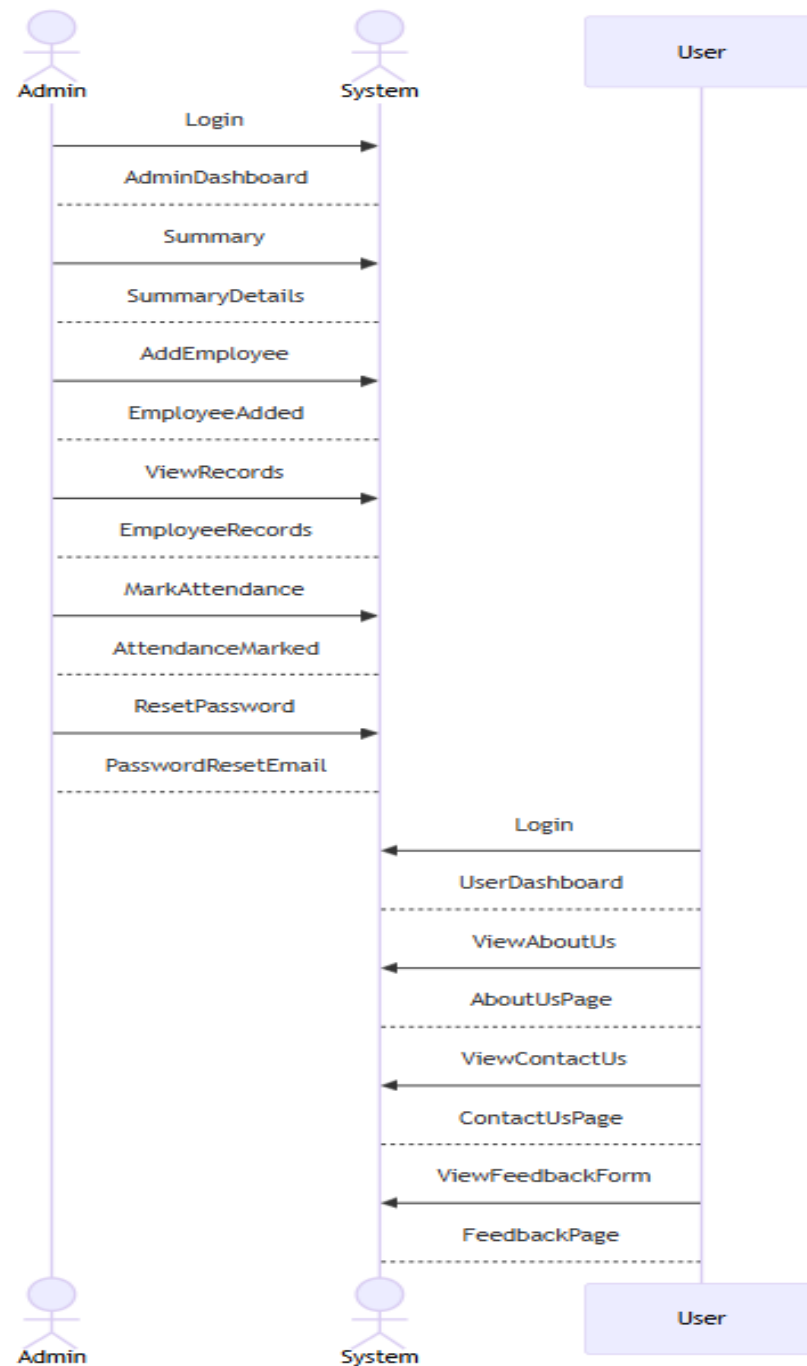


5. Flowcharts

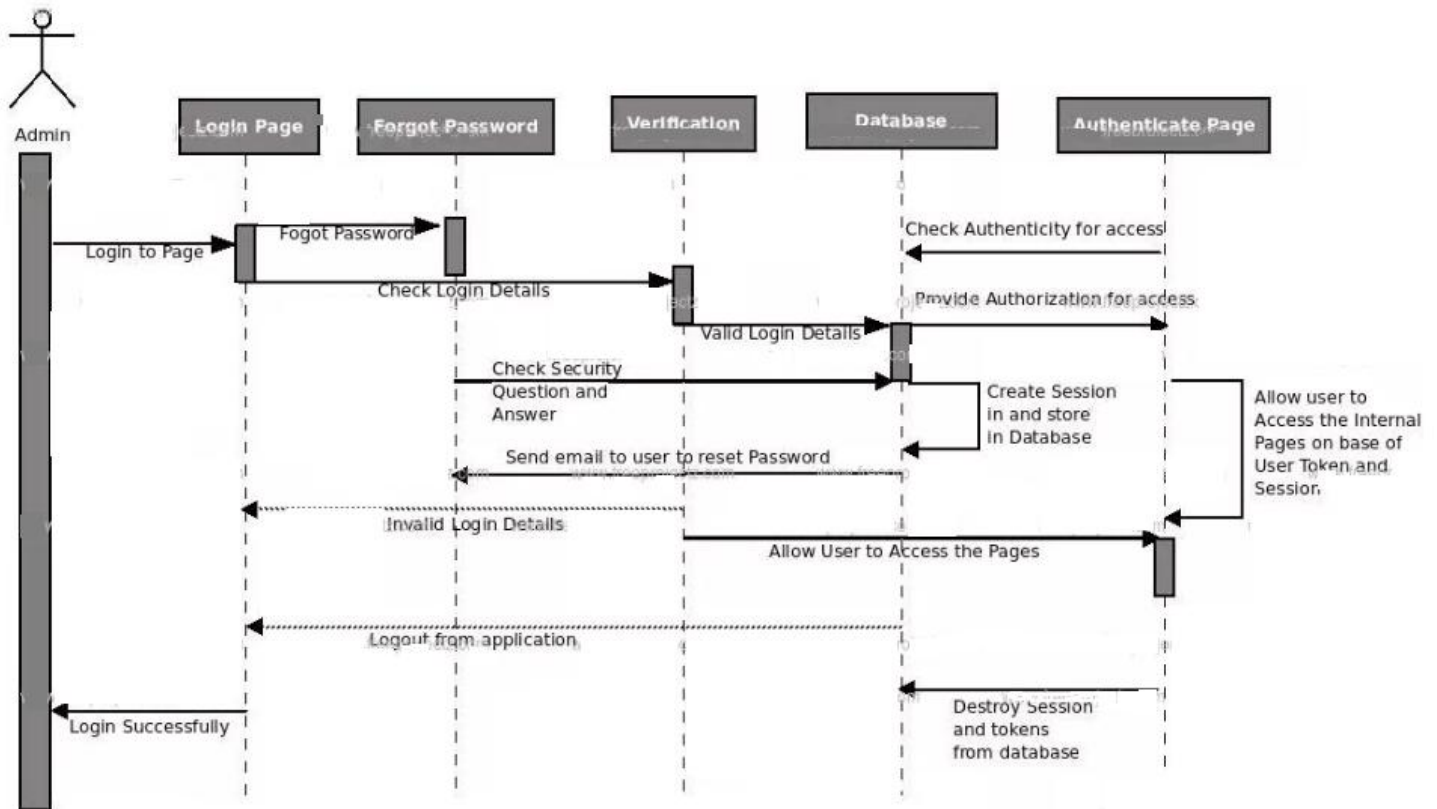




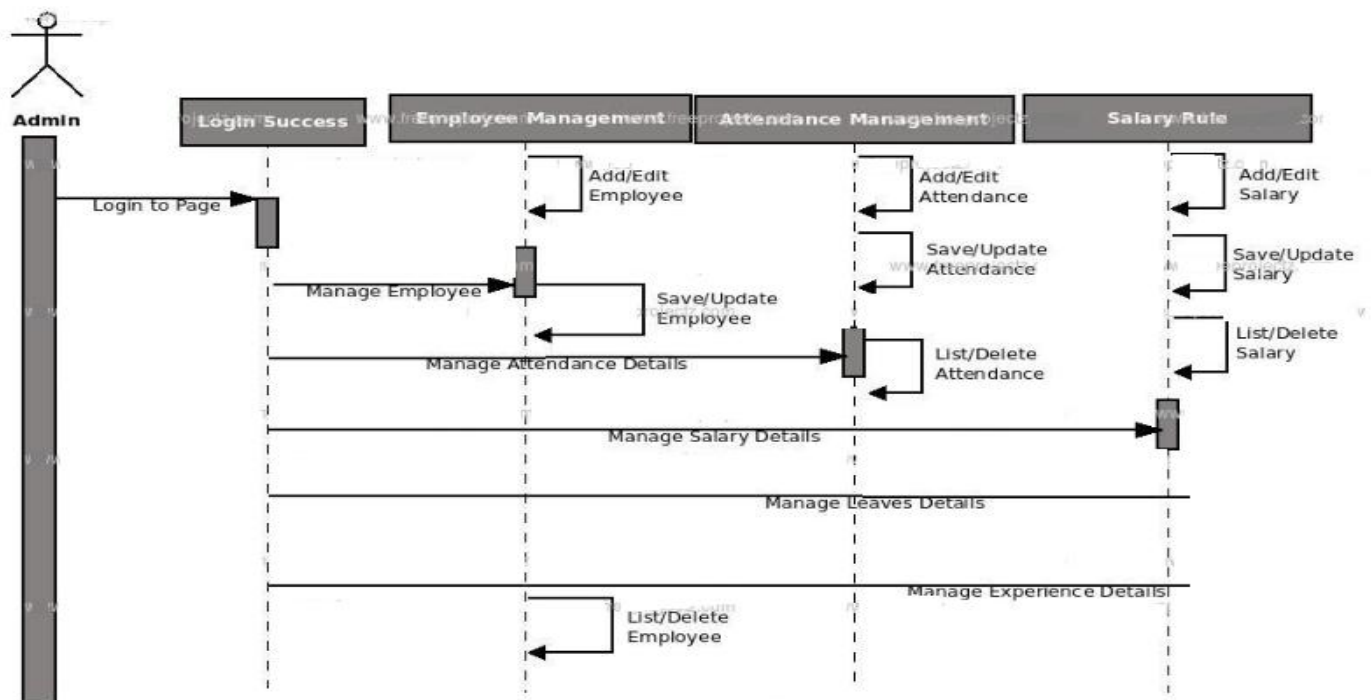
5. Sequence Diagrams



Login Sequence Diagram



Admin Sequence Diagram

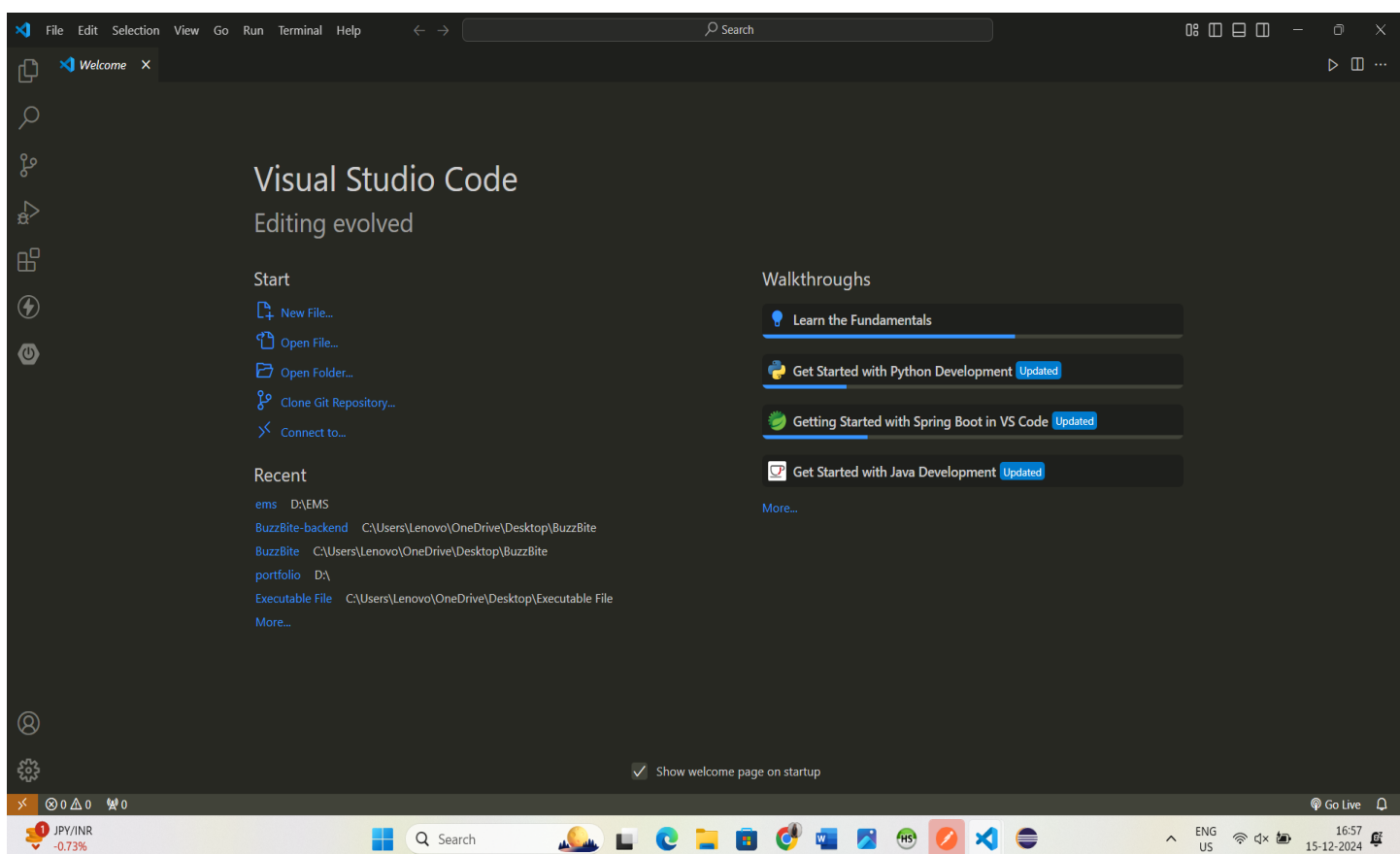


Chapter-04

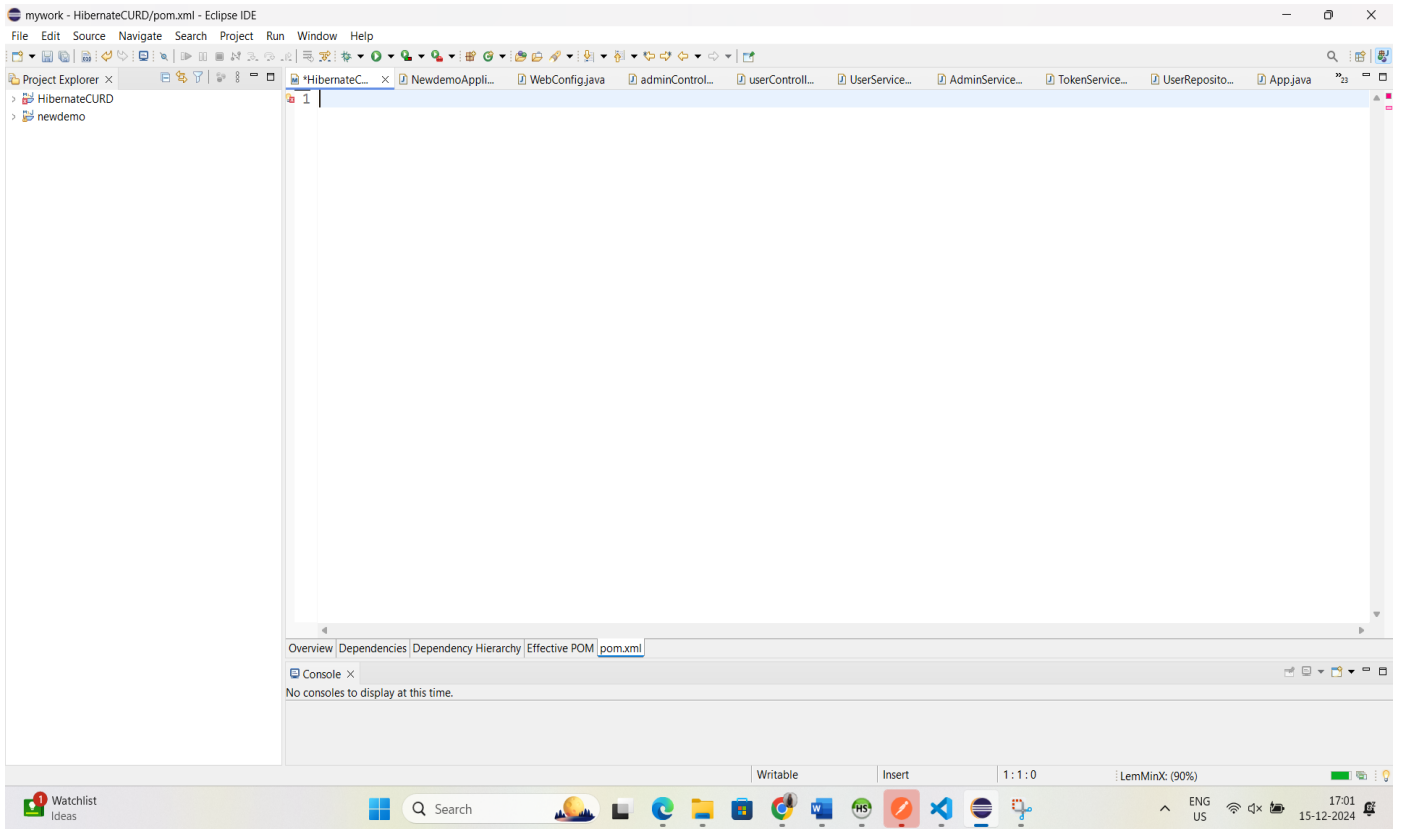
CODING

4.1 IDEs Used in the Project

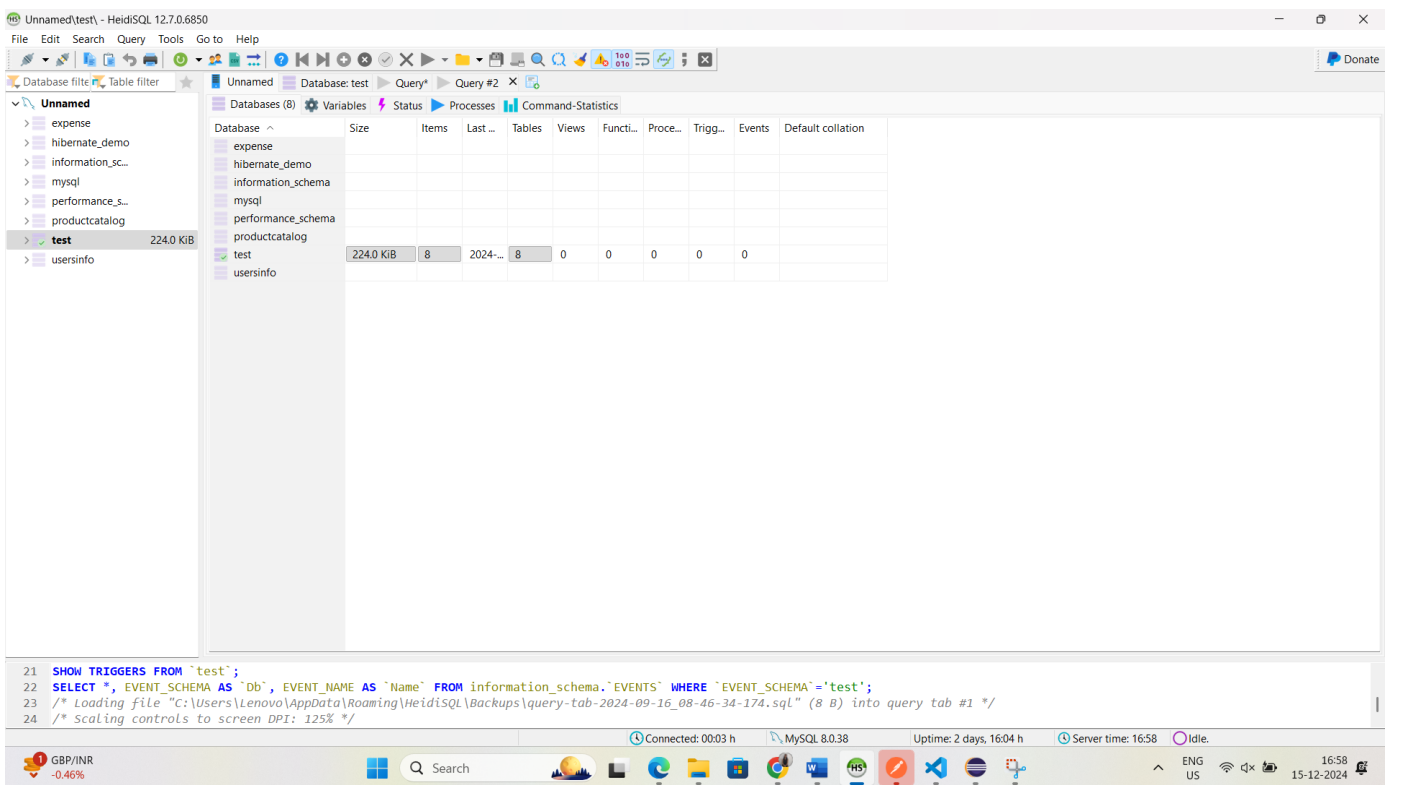
1. Frontend: Visual Studio Code (VS Code)



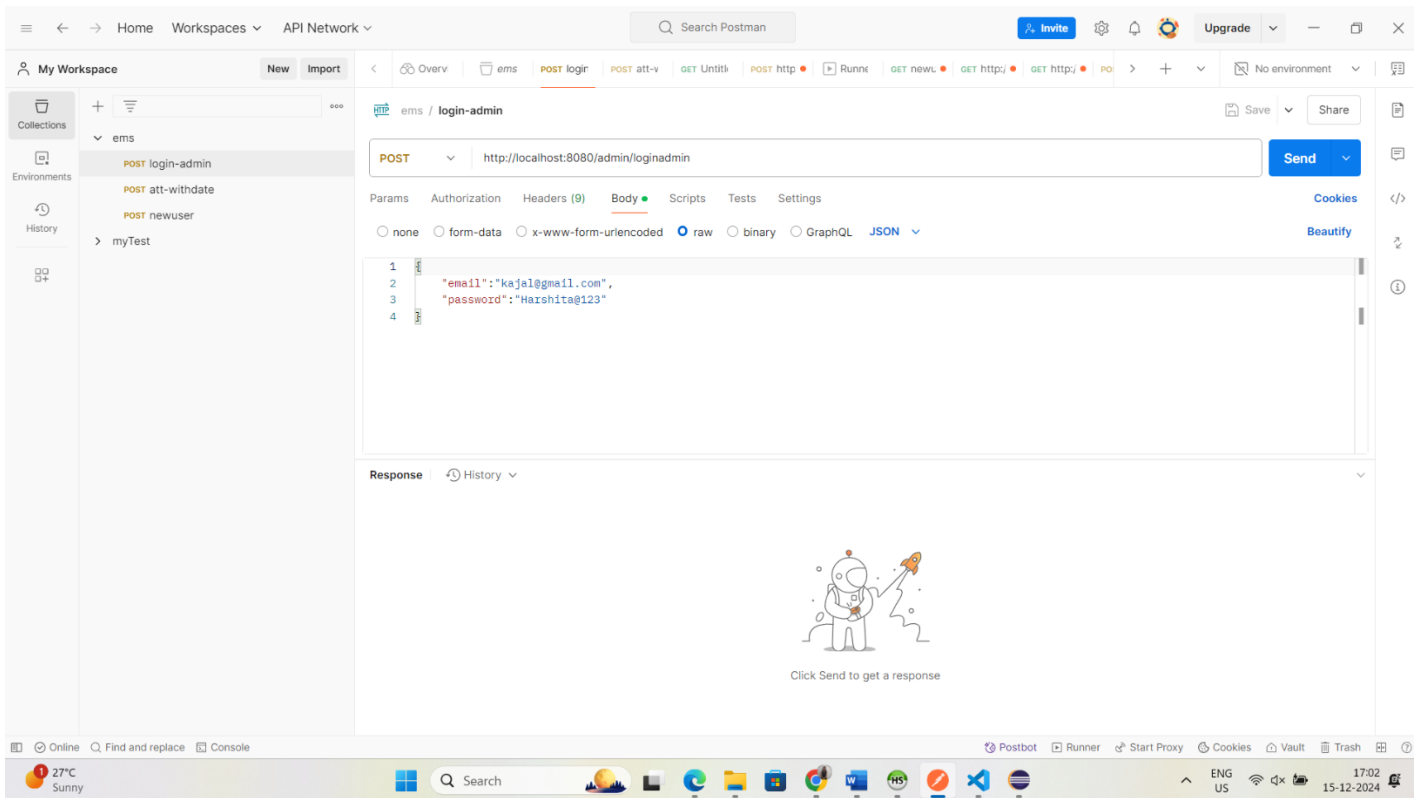
2. Backend: Eclipse IDE



3. Database Management: HeidiSQL



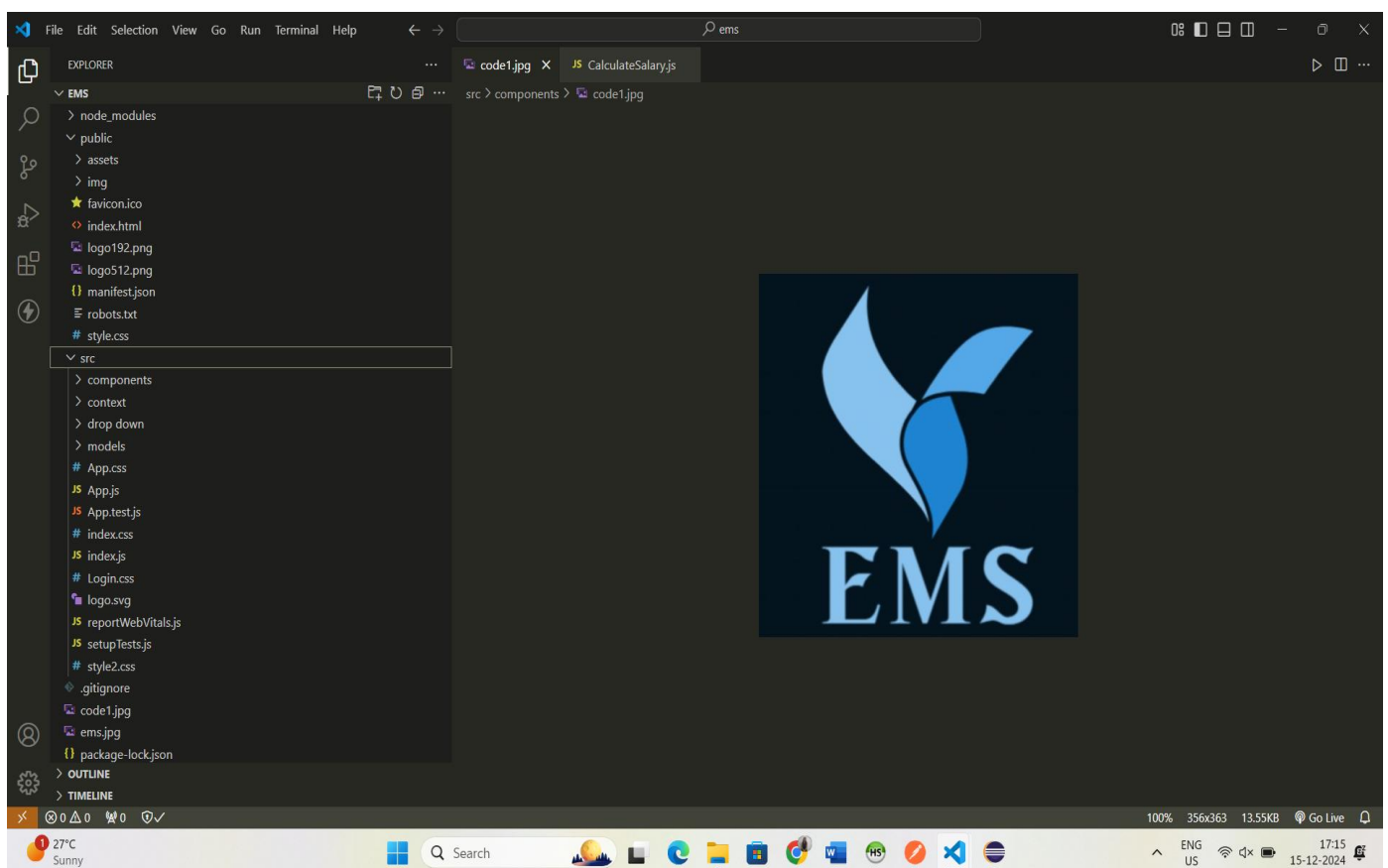
4. API Testing: Postman



4.2 Coding Part

The front-end, built in React, handles the user interface and sends requests to the back-end via REST APIs. The back-end, developed with Spring Boot, processes these requests and communicates with the database.

1.Folder Structure for Frontend (React Js)



1.Rounting (App.js)

Routing page: This is the main _app.js page which is responsible for rounting and rendering all the pages that are in pages folder to the user.

```
src > JS App.js > App
1  import 'bootstrap/dist/css/bootstrap.min.css';
2
3  import { BrowserRouter, Routes, Route } from "react-router-dom";
4  import Home from "../components/Home";
5  import Login from "../components/Login";
6  import NoPage from "../components/NoPage";
7  import Footer from "../components/Footer";
8  import ReactNav from "../components/ReactNav";
9  import DashboardOptions from "../components/DashboardOptions";
10 import About from "../components/About";
11 import Contact from "../components/Contact";
12 import SignUp from "../components/SignUp";
13 import Feedback from "../components/Feedback";
14 import Display from "../components/Display";
15
16 import ViewDashBoardUser from "../components/ViewDashBoardUser";
17 import CreateAttendance from "../components/CreateAttendance";
18 import Company from "../components/Company";
19 import Createrate from "../components/Createrate";
20 import CalculateSalary from "../components/CalculateSalary";
21 import GetAttendanceSummary from "../components/GetAttendanceSummary";
22 import ViewMessage from "../components/ViewMessage";
23 import ViewFeedback from "module:d:/EMS/ems/src/components/CreateEmp";
24 import CreateEmp from "../components/CreateEmp";
25 // import UserDashboard from "../components/UserDashBoard/UserDashBoard";
26
27
28
29 function App() {
30   return (
31     <>
32     <BrowserRouter>
33     <ReactNav />
34     <Routes>
35
36     <Route path="/" element={<Home />}></Route>
37     <Route path="/login" element={<Login />}></Route>
```

Ln 45, Col 78 (65 selected) Spaces: 2 UTF-8 LF {} JavaScript Go Live

```

src > JS App.js > App
29 function App() {
36     <Route path="/" element={<Home />}></Route>
37     <Route path="/login" element={<Login />}></Route>
38     <Route path="/dashboard" element={<DashboardOptions />}></Route>
39     <Route path="/about" element={<About /> />
40     <Route path="/contact" element={<Contact /> />
41     <Route path="/signup" element={<SignUp />}></Route>
42     <Route path="/createemp" element={<CreateEmp />}></Route>
43     <Route path="/feedback" element={<Feedback />}></Route>
44     <Route path="/display" element={<Display />}></Route>
45     /* <Route path="/userdashboard" element={<UserDashboard />}></Route> */
46
47     <Route path="/viewdashboard" element={<ViewDashBoardUser />}></Route>
48     <Route path="/markattendance" element={<CreateAttendance />}></Route>
49     <Route path="/companystatics" element={<Company />}></Route>
50     <Route path="/createrate" element={<Createrate />}></Route>
51     <Route path="/calculateTotalRating" element={<CalculateSalary />}></Route>
52     <Route path="/getsummary" element={<GetAttendanceSummary />}></Route>
53     <Route path="/getmessage" element={<ViewMessage />}></Route>
54     <Route path="/getfeedback" element={<ViewFeedback />}></Route>
55     <Route path="/signup" element={<ViewFeedback />}></Route>
56     <Route path="/createEmp" element={<CreateEmp />}></Route>
57     /* <Route path="/edituser" element={<EditUserModel />}></Route> */
58
59
60
61
62     <Route path="/*" element={<NoPage />}></Route>
63 </Routes>
64 <Footer />
65 </BrowserRouter>
66 </>
67 );
68 }
69
70 export default App;
71

```

2.index.html page

This is the HTML boilerplate for a React application created using Create React App (CRA)

```

1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="utf-8" />
6   <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
7   <meta name="viewport" content="width=device-width, initial-scale=1" />
8   <meta name="theme-color" content="#000000" />
9   <meta name="description" content="Web site created using create-react-app" />
10  <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
11  <link rel="stylesheet" href="/style.css">
12
13  <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
14  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css" rel="stylesheet"
15    integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOMLAsjC" crossorigin="anonymous">
16  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.10.0/font/bootstrap-icons.css">
17  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.6.0/css/all.min.css"
18    integrity="sha512-Kc323vGBEqzTmouAECnVceyQyqdsSiqlLQISBL29aUW4U/M7pSPA/gEUZQqv1cwx4OnYxTxve5UMg5GT6L4JJg=="
19    crossorigin="anonymous" referrerpolicy="no-referrer" />
20
21  <title>React App</title>
22 </head>
23
24 <body>
25   <noscript>You need to enable JavaScript to run this app.</noscript>
26   <div id="root"></div>
27
28 </body>
29 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"
30   integrity="sha384-MrcW6MFYlZcLA8N1+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxXm" crossorigin="anonymous"></script>
31
32 </html>

```

3.style.css

All the components css has been written in this file.

It defines the appearance and behavior of different sections like home pages, attendance forms, rate management forms, dashboards, tables, buttons, and containers.

```
* {
  margin: 0;
  padding: 0;
  /* font-family: Cambria, Cochin, Georgia, Times, 'Times New Roman', serif; */
  font-family: 'Times New Roman', Times, serif;
}

.home-container {
  background-image: url("/img/ems2.jpg");
  background-size: 100%;

  background-position: center center; /* Center the image */
  background-repeat: no-repeat; /* Ensure the image does not repeat */
  height: 100vh; /* Full height of the viewport */
  width: 100%; /* Full width of the container */
  display: flex;
  justify-content: center; /* Center content horizontally */
  align-items: center; /* Center content vertically */
  background-color: #c4c1c6; /* Light background color as a fallback */
  color: #333; /* Dark text color */
  text-align: center; /* Center text */
  filter: brightness(1);
}

#atten .container {
  width: 80%;
  margin: 10vh auto;
  padding: 20px;
  background-color: rgba(237, 236, 236, 0.975);
  border-radius: 10px;
  box-shadow: 0 4px 10px rgba(22, 22, 22, 0.975);
}

#atten h2 {
  font-size: 24px;
  color: #333;
}
```

4.Home.js page

Home page is a React component using framer-motion to create a fade-in effect on hover, displaying a welcoming message for an Employee Management System

```
import React from 'react';
import { motion } from 'framer-motion';

const Home = () => {
  return (
    <div className="home-container">
      <motion.div
        className="home-content"
        initial={{ opacity: 0 }} // Start hidden
        whileHover={{ opacity: 1 }} // Appear when hovered
        transition={{ duration: 0.5 }} // Duration of the animation
        style={{ backgroundColor: '■ rgba(230, 230, 250, 1)' }}
      >
        <h1>Welcome to the Employee Management System</h1>
        <p>
          Manage employee attendance and salaries effortlessly.
        </p>
      </motion.div>
    </div>
  );
};

export default Home;
```

5.About-us .js page

About page is a React component showcasing an "About Us" section with styled text and an image, describing the project's purpose of managing employee attendance and salaries efficiently

```
import React from 'react';

const About = () => {
  const textStyle = {
    fontSize: "1.8em",
    color: "lavender",
    fontStyle: "italic",
  };

  return (
    <div className="aboutus-card">
      <div className="image-box">
        
        <div className="content">
          <h2 style={{ fontSize: "2.2em", color: "lavender", fontStyle: "italic" }}>
            "Welcome to our project!!"
          </h2>
          <p style={{ fontSize: "1.8em" }} >
            Our project aims to efficiently manage employee attendance and related functionalities.
            We have developed a
          </p>
          <p style={{ fontSize: "1.8em" }} >
            user-friendly interface with features like marking attendance and calculating salaries.
          </p>
        </div>
      </div>
    </div>
  );
};

export default About;
```

6. Dashboard.js

DashboardOptions component provides a user-friendly dashboard with clickable cards for various actions like creating a new user, displaying users, marking attendance, and more. Each card has a confirmation dialog before navigation.

```
import React, { useEffect } from 'react';
import '../style2.css'; // Import if needed
import { useNavigate } from 'react-router-dom';
import Footer from './Footer';

function DashboardOptions() {
  const navigate = useNavigate();
  const handlecreateEmpRedirect = () => {
    const isConfirm = window.confirm("Are you sure to create new employee?");
    if (isConfirm) {
      navigate('/createEmp');
    }
  };
  const handledisplayRedirect = () => {
    const isConfirm = window.confirm("Are you sure to display all employee?");
    if (isConfirm) {
      navigate('/display');
    }
  };
  const handlemarkattendanceRedirect = () => {
    const isConfirm = window.confirm("Do you want to mark attendance of employee?");
    if (isConfirm) {
      navigate('/markattendance');
    }
  };
  const handlecompany = () => {
    const isConfirm = window.confirm("Do you want to view company statics?");
    if (isConfirm) {
      navigate('/companystatics');
    }
  };
  const handlecreaterate = () => {
    const isConfirm = window.confirm("Do you want to view company statics?");
    if (isConfirm) {
      navigate('/createrate');
    }
  };
};
```

7.Login.js

The component Login handles user login functionality, navigating to a dashboard or signup page based on login status.

```
import React, { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import axios from 'axios';
import { toast } from 'react-toastify';
const Login = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const navigate = useNavigate();

  const handleEmail = (e) => {
    setEmail(e.target.value);
  }
  const handlePassword = (e) => {
    setPassword(e.target.value);
  }

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      // Making a POST request to the backend using Axios
      const response = await axios.post('http://localhost:8080/admin/loginadmin', {
        email: email,
        password: password
      });
      localStorage.setItem('token', response.data.token);
      localStorage.setItem('isAdmin', 'true');
      console.log('Login Successful:', response.data);
      toast.success("You have successfully Login!");
      localStorage.setItem("isAdmin",true);
      navigate('/dashboard');
    } catch (error) {
      localStorage.removeItem('token');
      localStorage.removeItem('id');
      console.error('Error during login:', error.response ? error.response.data : error.message);
      alert('Login failed. Please check your credentials and try again.');
```


8.SignUp.js page

SignUp page allows users to create a new account by filling out a form

```
import React, { useState } from 'react';
import axios from 'axios';
import { toast } from 'react-toastify';

const SignUp = () => {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    password: '',
    role: '',
    showPassword: false,
  });
  const [errors, setErrors] = useState({});
  const [errorMessage, setErrorMessage] = useState('');

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  };

  const toggleShowPassword = () => {
    setFormData({ ...formData, showPassword: !formData.showPassword });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      const response = await axios.post('http://localhost:8080/admin/newadmin', {
        name: formData.name,
        email: formData.email,
        password: formData.password,
        role: formData.role,
      });
      console.log('User Created:', response.data);
      setErrorMessage('');
      setErrors({});
    }
  }
}
```

9.Contact.js page

The Contact component allows users to submit a contact form with their name, email, and message.

```
import React, { useState } from 'react';

const Contact = () => {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    message: ''
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    // Send form data to the backend API
    try {
      const response = await fetch('http://localhost:8080/admin/contact', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify(formData), // Send form data as JSON
      });

      if (response.ok) {
        console.log('Form submitted successfully');
        alert('Your message has been sent successfully!');
      } else {
        console.error('Failed to submit the form');
        alert('There was an error submitting your form. Please try again.');
```

10.Feedback.js page

The Feedback component allows users to submit feedback through a form

```
import React, { useState } from 'react';
import axios from 'axios';
const Feedback = () => {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    rating: '',
    feedback: '',
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      // Send the feedback data to the backend API
      const response = await axios.post('http://localhost:8080/admin/feedback', formData);

      // Log the response and reset the form if successful
      console.log('Feedback Submitted:', response.data);
      setFormData({ name: '', email: '', rating: '', feedback: '' });
      alert('Feedback submitted successfully!');
    } catch (error) {
      console.error('Error submitting feedback:', error);
      alert('Failed to submit feedback. Please try again.');
```

11.ReactNavBar.js page

The Navbar component is a responsive navigation bar for a web application that handles user authentication, profile management, navigation, and logout functionality, with specific UI adjustments for admin and non-admin users.

```
import React, { useEffect, useState, useRef } from 'react';
import { useNavigate } from 'react-router-dom';

function Navbar() {
  const navigate = useNavigate();
  const codeLogo = './img/code1.jpg';
  const checkBoxRef = useRef(null);
  const navRef = useRef(null);

  const [isAdmin, setIsAdmin] = useState(false);
  const [userName, setUserName] = useState('');
  const [profileImage, setProfileImage] = useState('');
  const [showDetails, setShowDetails] = useState(false);

  useEffect(() => {
    // Load user data from localStorage
    const loadUserData = () => {
      const adminStatus = localStorage.getItem('isAdmin') === 'true';
      const storedUserName = localStorage.getItem('userName') || 'Harshita Bajiya';
      const storedProfileImage =
        localStorage.getItem('profileImage') || 'https://via.placeholder.com/40';

      setIsAdmin(adminStatus);
      setUserName(storedUserName);
      setProfileImage(storedProfileImage);
    };

    loadUserData();

    // Listen for storage changes to update user data dynamically
    const handleStorageChange = () => loadUserData();
    window.addEventListener('storage', handleStorageChange);

    return () => window.removeEventListener('storage', handleStorageChange);
  }, []);

  const handleNavigation = (path) => {
```

```

const handleNavigation = (path) => {
  navigate(path);
};

const handleLogout = () => {
  localStorage.clear(); // Clear all user-related data
  setIsAdmin(false);
  navigate('/login'); // Redirect to login page
};

const toggleDetails = () => {
  setShowDetails(!showDetails);
};

return (
  <nav ref={navRef}>
    <input type="checkbox" id="check" ref={checkBoxRef} name="" value="" />
    <label htmlFor="check" id="checkbtn">
      <i className="fa fa-bars"></i>
    </label>
    <label className="logo">
      <img className="logo-img" src={codeLogo} alt="Logo" />
    </label>

    <span className="login-container">
      {!isAdmin ? (
        <>
          <a className="login-btn" id="loginBtn" onClick={() => navigate('/login')}>
            Login
          </a>
          <a className="sign-btn" id="signBtn" onClick={() => navigate('/signup')}>
            Sign Up
          </a>
        </>
      ) : (

```

```

      <span className="login-container">...
    </span>

    <ul>
      <li>
        <a onClick={() => handleNavigation('/')}>Home</a>
      </li>
      <li>
        <a onClick={() => handleNavigation('/about')}>About Us</a>
      </li>
      {isAdmin ? (
        <li>
          <a onClick={() => handleNavigation('/dashboard')}>Dashboard</a>
        </li>
      ) : (
        <li>
          <a ...
          </a>
        </li>
      )}
      <li>...
    </li>
    { /* <li>
      <a onClick={() => handleNavigation('/userdashboard')}> Us</a>
    </li> */ }
    <li>...
  </li>
</ul>

</nav>
);

```

```
export default Navbar;
```

12.Footer.js page

The Footer component contains contact information, quick links, social media links, and a copyright notice.

```
import React from 'react';
import { FaFacebook, FaGithub, FaTwitter, FaLinkedin, FaEnvelope, FaPhone } from 'react-icons/fa';

const Footer = () => {
  return (
    <footer id='footer.'>
      <div className="container">
        <div className="footer-content">...
      </div>
      <div className="footer-content">...
      </div>
      <div className="footer-content">
        <h3>Social Links</h3>
        <ul className="social-icons">
          <li style={{ marginRight: '12px' }}>
            <a href="https://www.facebook.com/yourprofile" target="_blank" rel="noopener noreferrer">
              <FaFacebook />
            </a>
          </li>
          <li style={{ marginRight: '12px' }}>...
          </li>
          <li style={{ marginRight: '12px' }}>
            <a href="https://twitter.com/yourprofile" target="_blank" rel="noopener noreferrer">...
            </a>
          </li>
          <li>...
          </li>
        </ul>
      </div>
      <div>
        <div className="bottom-bar">...
      </div>
    </footer>
  );
};

export default Footer;
```

2.API Testing (Postman)

1.login admin api

<http://localhost:8080/admin/loginadmin>

The screenshot displays the Postman interface for a POST request to `http://localhost:8080/admin/loginadmin`. The request body is a JSON object with the following fields:

```
1 {
2   "email": "kajal@gmail.com",
3   "password": "Harshita@123"
4 }
```

The response is a 200 OK status, received in 2.30 seconds with a body size of 586 B. The response body is a JSON object with the following fields:

```
1 {
2   "id": 1,
3   "name": "Kajal Kumari",
4   "email": "kajal@gmail.com",
5   "role": "Frontend Developer",
6   "createdAt": "2024-11-11",
7   "updatedAt": "2024-11-11",
8   "status": null,
9   "token": "eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJEsIn1YiI6ImthamFsQGdtYWlsLmNvbSIsIm1hdCI6MTczNDI2ODIwOSwiZXhwIjozNzY0MzA0MjA5fQ.ihIBj-LUAKRsdDErV00aEo1rNtfaEBCs6n30ryeYs-M",
10  "message": null
11 }
```

A Postbot notification is visible at the bottom right, indicating the keyboard shortcut `Ctrl Alt P`.

2.get user by id (api)

<http://localhost:8080/admin/users/1>

The screenshot shows a REST client interface. At the top, the method is **GET** and the URL is `http://localhost:8080/admin/users/1`. A **Send** button is on the right. Below the URL bar, tabs for **Params**, **Authorization**, **Headers (10)**, **Body**, **Scripts**, **Tests**, and **Settings** are visible. The **Authorization** tab is active, showing **Auth Type** as **Bearer Token** and a **Token** value: `eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJEsln...`. A note states: "The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization." Below this, the **Body** tab is active, displaying a JSON response in **Pretty** format:

```
1 {
2   "id": 1,
3   "name": "Harshita Bajiya",
4   "email": "harshita123@gmail.com",
5   "role": "user",
6   "createdAt": "2024-09-27",
7   "updatedAt": "2024-11-11",
8   "status": "Active",
9   "message": "User data successfully retrieved"
10 }
```

 The status bar at the bottom indicates **200 OK**, **55 ms**, and **445 B**. There are also icons for **Save Response** and **Cookies**.

3.get all users (api)

<http://localhost:8080/admin/allusers>

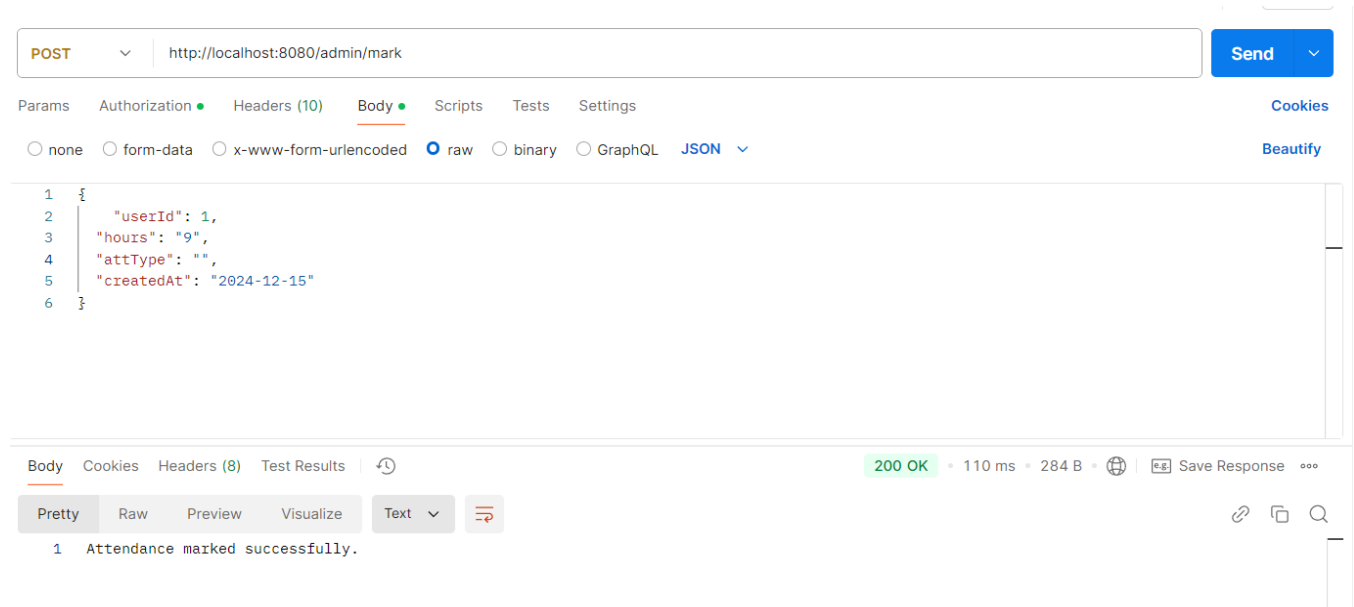
The screenshot shows a REST client interface. At the top, the method is **GET** and the URL is `http://localhost:8080/admin/allusers`. A **Send** button is on the right. Below the URL bar, tabs for **Params**, **Authorization**, **Headers (10)**, **Body**, **Scripts**, **Tests**, and **Settings** are visible. The **Authorization** tab is active, showing **Auth Type** as **Bearer Token** and a **Token** value: `eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJEsln...`. A note states: "The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization." Below this, the **Body** tab is active, displaying a JSON response in **Pretty** format:

```
22 > { ...
31 },
32 {
33   "id": 10,
34   "name": "abcd deff",
35   "email": "de@gmail.com",
36   "role": "Frontend Developer",
37   "createdAt": "2024-11-12",
38   "updatedAt": "2024-12-05",
39   "status": "Active",
40   "message": "User data successfully retrieved"
41 }
```

 The status bar at the bottom indicates **200 OK**, **52 ms**, and **1.01 KB**. There are also icons for **Save Response** and **Cookies**.

4.mark attendance(api)

<http://localhost:8080/admin/mark>



POST ▼ http://localhost:8080/admin/mark Send ▼

Params Authorization ● Headers (10) **Body** ● Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▼ Beautify

```
1 {
2   "userId": 1,
3   "hours": "9",
4   "attType": "",
5   "createdAt": "2024-12-15"
6 }
```

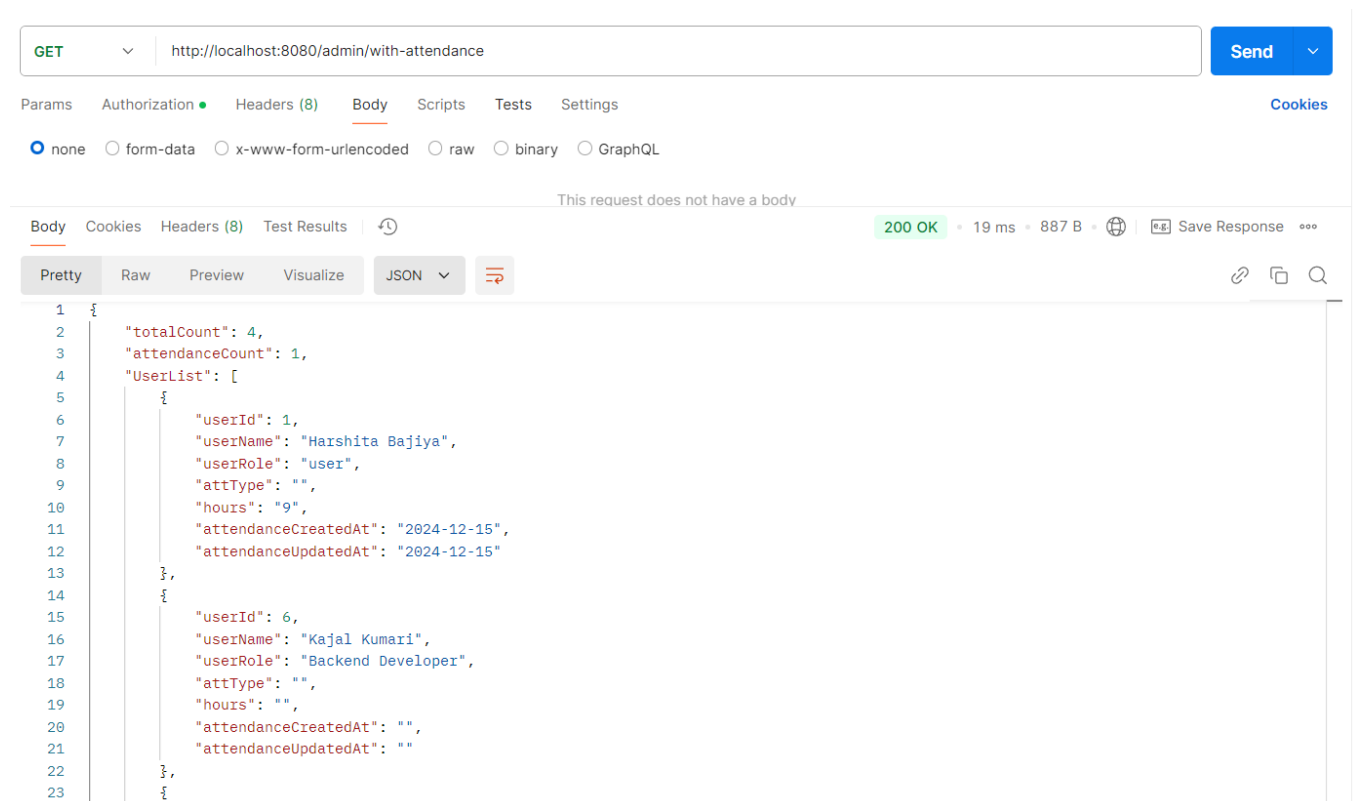
Body Cookies Headers (8) Test Results 🔄 200 OK • 110 ms • 284 B • 🌐 💾 Save Response ⋮

Pretty Raw Preview Visualize Text ▼ 🔗 📄 🔍

1 Attendance marked successfully.

5. getAllUsersWithAttendance (api)

<http://localhost:8080/admin/with-attendance>



GET ▼ http://localhost:8080/admin/with-attendance Send ▼

Params Authorization ● Headers (8) **Body** ● Scripts Tests Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

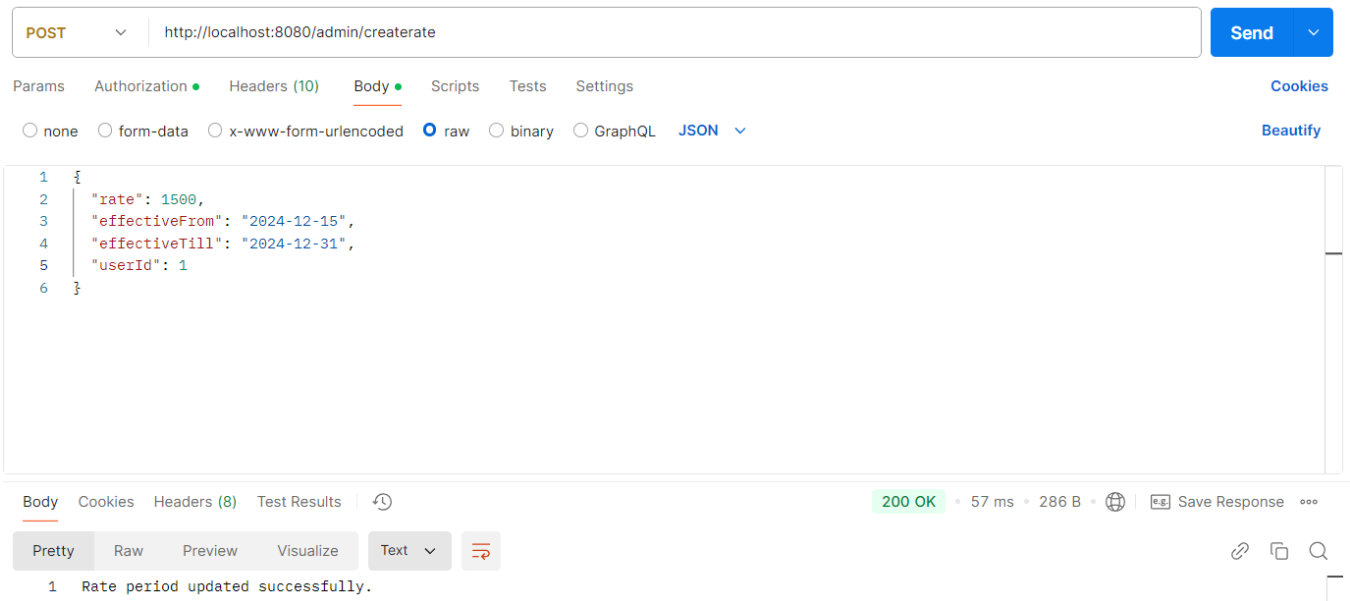
Body Cookies Headers (8) Test Results 🔄 200 OK • 19 ms • 887 B • 🌐 💾 Save Response ⋮

Pretty Raw Preview Visualize JSON ▼ 🔗 📄 🔍

```
1 {
2   "totalCount": 4,
3   "attendanceCount": 1,
4   "UserList": [
5     {
6       "userId": 1,
7       "userName": "Harshita Bajiya",
8       "userRole": "user",
9       "attType": "",
10      "hours": "9",
11      "attendanceCreatedAt": "2024-12-15",
12      "attendanceUpdatedAt": "2024-12-15"
13    },
14    {
15      "userId": 6,
16      "userName": "Kajal Kumari",
17      "userRole": "Backend Developer",
18      "attType": "",
19      "hours": "",
20      "attendanceCreatedAt": "",
21      "attendanceUpdatedAt": ""
22    },
23  ]
24 }
```

6.Create Rate (api)

<http://localhost:8080/admin/createrate>



POST ▼ http://localhost:8080/admin/createrate Send ▼

Params Authorization ● Headers (10) Body ● Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▼ Beautify

```
1 {  
2   "rate": 1500,  
3   "effectiveFrom": "2024-12-15",  
4   "effectiveTill": "2024-12-31",  
5   "userId": 1  
6 }
```

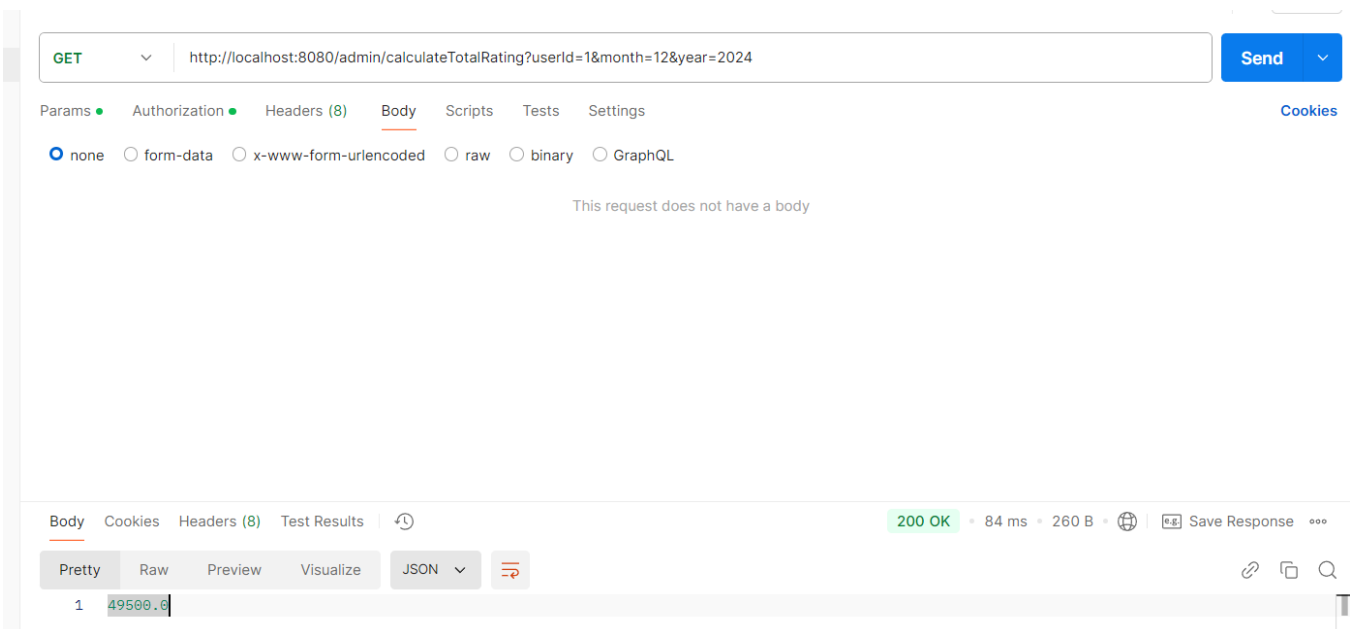
Body Cookies Headers (8) Test Results ↺ 200 OK • 57 ms • 286 B • 🌐 📄 Save Response ⋮

Pretty Raw Preview Visualize Text ▼ 🔗 📄 🔍

1 Rate period updated successfully.

7.Calculate Salary (api)

<http://localhost:8080/admin/calculateTotalRating?userId=1&month=12&year=2024>



GET ▼ http://localhost:8080/admin/calculateTotalRating?userId=1&month=12&year=2024 Send ▼

Params ● Authorization ● Headers (8) Body Scripts Tests Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (8) Test Results ↺ 200 OK • 84 ms • 260 B • 🌐 📄 Save Response ⋮

Pretty Raw Preview Visualize JSON ▼ 🔗 📄 🔍

1 49500.0

7. getRatesByUserAndYear (api)

<http://localhost:8080/admin/1/year/2024>

GET ▼ http://localhost:8080/admin/1/year/2024 Send ▼

Params Authorization ● Headers (8) Body Scripts Tests Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (8) Test Results 🕒 200 OK • 65 ms • 289 B • 🌐 💾 Save Response ⋮

Pretty Raw Preview Visualize JSON ▼ 🔗 📄 🔍

```
1 [
2   {
3     "month": "DECEMBER",
4     "rate": 1500.0
5   }
6 ]
```

8. Display Feedback message(api)

<http://localhost:8080/admin/getfeedback>

GET ▼ http://localhost:8080/admin/getfeedback Send ▼

Params Authorization ● Headers (8) Body Scripts Tests Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (8) Test Results 🕒 200 OK • 158 ms • 931 B • 🌐 💾 Save Response ⋮

Pretty Raw Preview Visualize JSON ▼ 🔗 📄 🔍

```
1 [
2   {
3     "id": 1,
4     "name": "John Doe",
5     "email": "john.doe@example.com",
6     "rating": 5,
7     "feedback": "Great service!"
8   },
9   {
10    "id": 2,
11    "name": "Harshita Bajiya",
12    "email": "harshitabajiya2838@gmail.com",
13    "rating": 4,
14    "feedback": "good"
15  },
16  {
17    "id": 3,
18    "name": "john",
19    "email": "john@gmail.com",
20    "rating": 5,
21    "feedback": "Great !!"
22  }
23 ]
```

3.Database folder Structure (HiediSQL)

Unnamed(testuser) - HeidiSQL 12.7.0.6850

File Edit Search Query Tools Go to Help

Database filter Table filter

testuser: 4 rows total (exact)

#	id	created_at	email	name	password	role	status	updated_at
1	1	2024-09-27	harshita123@gmail.com	Harshita Bajiya	\$2a\$10\$VksngBZqzt1JbsAlpFqvvubhg23j2ChY...	user	Active	2024-11-11
2	6	2024-11-12	kajal@gmail.com	Kajal Kumari	\$2a\$10\$z34/koelA6uJazX7Q432u002KSQUEP...	Backend Developer	Active	2024-12-11
3	7	2024-11-12	rain@gmail.com	Ruchi Jain	\$2a\$10\$WPHMFQ10QVB.1Rv5ZDZK8.LBHqS8m...	Frontend Developer	Active	2024-12-05
4	10	2024-11-12	de@gmail.com	abcd deff	\$2a\$10\$aM.WFKCJ8ond97WcFLU.w.OPYYk1G...	Frontend Developer	Active	2024-12-05

130 SELECT * FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA='test' AND TABLE_NAME='user' AND REFERENCED_TABLE_NAME IS NOT NULL;

131 SHOW CREATE TABLE `test`.`user`;

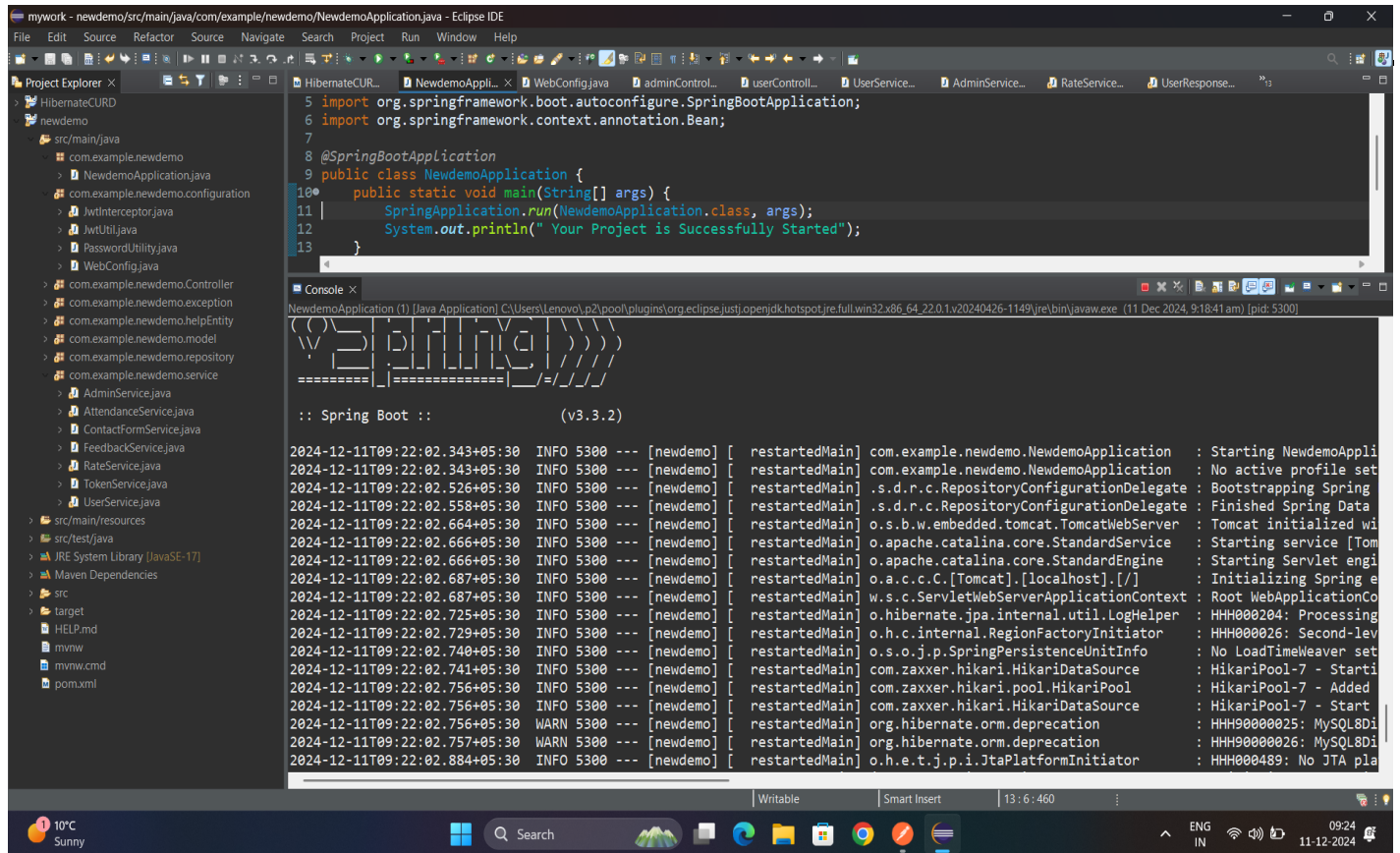
132 SELECT tc.CONSTRAINT_NAME, cc.CHECK_CLAUSE FROM `information\_schema`.`CHECK\_CONSTRAINTS` AS cc, `information\_schema`.`TABLE\_CONSTRAINTS` AS tc WHERE tc.CONSTRAINT_SCHEMA='test' AND tc

133 SELECT * FROM `test`.`user` LIMIT 1000;

r1: c2 Connected: 04:01 h MySQL 8.0.38 Uptime: 2 days, 20:02 h Server time: 20:56 Idle.

4. Folder Structure for Backend

(SpringBoot)



```
mywork - newdemo/src/main/java/com/example/newdemo/NewdemoApplication.java - Eclipse IDE
File Edit Source Refactor Source Navigate Search Project Run Window Help

Project Explorer
newdemo
src/main/java
com.example.newdemo
NewdemoApplication.java
com.example.newdemo.configuration
JwtInterceptor.java
JwtUtil.java
PasswordUtility.java
WebConfig.java
com.example.newdemo.Controller
com.example.newdemo.exception
com.example.newdemo.helpEntity
com.example.newdemo.model
com.example.newdemo.repository
com.example.newdemo.service
AdminService.java
AttendanceService.java
ContactFormService.java
FeedbackService.java
RateService.java
TokenService.java
UserService.java
src/main/resources
src/test/java
JRE System Library (JavaSE-17)
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
pom.xml

NewdemoApplication.java
5 import org.springframework.boot.autoconfigure.SpringBootApplication;
6 import org.springframework.context.annotation.Bean;
7
8 @SpringBootApplication
9 public class NewdemoApplication {
10     public static void main(String[] args) {
11         SpringApplication.run(NewdemoApplication.class, args);
12         System.out.println(" Your Project is Successfully Started");
13     }
14 }

Console
NewdemoApplication (1) [Java Application] C:\Users\Lenovo\p2\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.22.0.1.v20240426-1149\jre\bin\javaw.exe (11 Dec 2024, 9:18:41 am) [pid: 5300]

:: Spring Boot :: (v3.3.2)

2024-12-11T09:22:02.343+05:30 INFO 5300 --- [newdemo] [ restartedMain] com.example.newdemo.NewdemoApplication : Starting NewdemoAppli
2024-12-11T09:22:02.343+05:30 INFO 5300 --- [newdemo] [ restartedMain] com.example.newdemo.NewdemoApplication : No active profile set
2024-12-11T09:22:02.526+05:30 INFO 5300 --- [newdemo] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring
2024-12-11T09:22:02.558+05:30 INFO 5300 --- [newdemo] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data
2024-12-11T09:22:02.664+05:30 INFO 5300 --- [newdemo] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized wi
2024-12-11T09:22:02.666+05:30 INFO 5300 --- [newdemo] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tom
2024-12-11T09:22:02.666+05:30 INFO 5300 --- [newdemo] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engi
2024-12-11T09:22:02.687+05:30 INFO 5300 --- [newdemo] [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring e
2024-12-11T09:22:02.687+05:30 INFO 5300 --- [newdemo] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationCo
2024-12-11T09:22:02.725+05:30 INFO 5300 --- [newdemo] [ restartedMain] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing
2024-12-11T09:22:02.729+05:30 INFO 5300 --- [newdemo] [ restartedMain] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-lev
2024-12-11T09:22:02.740+05:30 INFO 5300 --- [newdemo] [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver set
2024-12-11T09:22:02.741+05:30 INFO 5300 --- [newdemo] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-7 - Starti
2024-12-11T09:22:02.756+05:30 INFO 5300 --- [newdemo] [ restartedMain] com.zaxxer.hikari.pool.HikariPool : HikariPool-7 - Added
2024-12-11T09:22:02.756+05:30 INFO 5300 --- [newdemo] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-7 - Start
2024-12-11T09:22:02.756+05:30 INFO 5300 --- [newdemo] [ restartedMain] org.hibernate.orm.deprecation : HHH90000025: MySQL8Di
2024-12-11T09:22:02.757+05:30 WARN 5300 --- [newdemo] [ restartedMain] org.hibernate.orm.deprecation : HHH90000025: MySQL8Di
2024-12-11T09:22:02.884+05:30 INFO 5300 --- [newdemo] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA pla
```

Controller

Controllers play a pivotal role in handling incoming HTTP requests, orchestrating business logic, and determining the view or data to return

The Controller used in project are:

- 1.AdminController
- 2.UserController

1.AdminController

```
1 package com.example.newdemo.Controller;
2
3 import java.time.LocalDate;
4
52
53 @RestController
54 @RequestMapping("/admin")
55 public class adminController {
56
57     @Autowired
58     private AdminService adminService;
59     @Autowired
60     private ContactFormService contactFormService;
61
62     @Autowired
63     private AttendanceService attendanceService;
64     @Autowired
65     private RateService rateService;
66     @Autowired
67     private FeedbackService feedbackService;
68     @Autowired
69     private UserService userService;
70     @Autowired
71     private TokenService tokenService;
72
73
74     @PostMapping("/newadmin")
75     public ResponseAdmin createNewAdmin(@Valid @RequestBody AdminEntity adminEntity) {
76         return adminService.insertNewAdmin(adminEntity);
77     }
78     @PostMapping("/loginadmin")
79     public ResponseAdmin loginAdmin(@Valid @RequestBody LoginRequest loginRequest) {
80         System.out.println("Hi Rahul - Starting loginAdmin method");
81
82         // Authenticate user
83         ResponseAdmin responseAdmin = adminService.loginAdmin(loginRequest.getEmail(), loginRequest.getPassword());
84
85         if (responseAdmin != null) {
86             System.out.println("Admin authenticated successfully.");
87         } else {
88             System.out.println("Admin authentication failed.");
89         }
90
91         return responseAdmin;
92     }
93
94     //update user ka control mere admin ke pass hai
95     @PutMapping("/update/{id}")
96     public ResponseEntity<Object> updateEntity(
97         @PathVariable Long id,
98         @RequestBody UserEntity updatedEntity,
99         @RequestHeader("Authorization") String authorizationHeader) {
100
101         // Extract the token from the Authorization header
102         String token = authorizationHeader.replace("Bearer ", "");
103
104         // Validate the token
105         if (!tokenService.validateToken(token)) {
106             // Token is invalid or expired, return 401 Unauthorized
107             return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid or expired token");
108         }
109
110         try {
111             // Proceed to update the user without handling roles
112             ResponseUser updatedUser = adminService.updateUser(id, updatedEntity);
113         } catch (Exception e) {
114             // Handle exception
115         }
116     }
117 }
```

```

@GetMapping("/message")
public ResponseEntity<List<ContactForm>> getAllMessages() {
    List<ContactForm> messages = contactFormService.getAllMessages();
    return ResponseEntity.ok(messages);
}
@PostMapping("/feedback")
public ResponseEntity<Feedback> submitFeedback(@RequestBody Feedback feedback) {
    Feedback savedFeedback = feedbackService.saveFeedback(feedback);
    return new ResponseEntity<>(savedFeedback, HttpStatus.CREATED);
}
@GetMapping("/getfeedback")
public ResponseEntity<List<Feedback>> getAllFeedbacks() {
    List<Feedback> feedbacks = feedbackService.getAllFeedbacks();
    return new ResponseEntity<>(feedbacks, HttpStatus.OK);
}

private ResponseEntity<Object> buildErrorResponse(HttpStatus status, String message, String path) {
    Map<String, Object> errorDetails = new HashMap<>();
    errorDetails.put("status", status.value());
    errorDetails.put("message", message);
    errorDetails.put("path", path);
    return ResponseEntity.status(status).body(errorDetails);
}

private ResponseEntity<Object> buildSuccessResponse(HttpStatus status, String message, Object data) {
    Map<String, Object> successDetails = new HashMap<>();
    successDetails.put("status", status.value());
    successDetails.put("message", message);
    successDetails.put("data", data);
    return ResponseEntity.status(status).body(successDetails);
}

```

2.UserController

```

1 package com.example.newdemo.Controller;
2
3 *import com.example.newdemo.exception.ValidationUtil.UserNotFoundException;
42
43 @RestController
44 @RequestMapping("/user")
45 public class UserController {
46
47     @Autowired
48     private AdminService adminService;
49
50     @Autowired
51     private AttendanceService attendanceService;
52     @Autowired
53     private RateService rateService;
54
55     @Autowired
56     private UserService userService;
57     @Autowired
58     private TokenService tokenService;
59
60     @PostMapping("/newuser")
61     public ResponseUser createUser(@Valid @RequestBody UserEntity userEntity) {
62         return adminService.insertNewUser(userEntity);
63     }
64
65     @PostMapping("/loginuser")
66     public ResponseUser loginUser(@Valid @RequestBody LoginRequest loginRequest) {
67         System.out.println("Hi Rahul - Starting loginUser method");
68
69         // Authenticate user
70         ResponseUser responseUser = adminService.loginuser(loginRequest.getEmail(), loginRequest.getPassword());
71

```

3.WebConfig

This module connects the Frontend and the Backend part (integration)

Frontend port:3000

Backend port:8080

```
1 package com.example.newdemo.configuration;
2
3
4 import org.springframework.beans.factory.annotation.Autowired;
5
6
7 @Configuration
8 public class WebConfig implements WebMvcConfigurer {
9
10     @Autowired
11     private JwtInterceptor jwtInterceptor;
12
13     @Override
14     public void addInterceptors(InterceptorRegistry registry) {
15         registry.addInterceptor(jwtInterceptor).addPathPatterns("/admin/allusers");
16     }
17
18     @Bean
19     public CorsFilter corsFilter() {
20         // Create a new CorsConfiguration object and configure it
21         CorsConfiguration config = new CorsConfiguration();
22         config.setAllowCredentials(true);
23         config.addAllowedOrigin("http://localhost:3000");
24         config.addAllowedHeader("*");
25         config.addAllowedMethod("*");
26
27         // Set up the source and register the configuration
28         UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
29         source.registerCorsConfiguration("/*", config);
30
31         // Return the new CorsFilter
32         return new CorsFilter(source);
33     }
34 }
35
36
37
38
39
40
41 }
```


4.Repository

In Spring Boot, a repository is used to manage data persistence and retrieval in a database. It provides an abstraction layer over the database and simplifies the process of interacting with it, allowing developers to focus on business logic rather than database operations

The repository used in the project are:

1.AdminRepository.java

- It is responsible for data access for admin-related operations
- It manages CRUD operations for admin entities, such as fetching, updating, and deleting admin data

2. AttendanceRepository.java

- Responsible for managing attendance records.
- Queries related to user attendance, including date ranges and specific user attendance
- Provides methods to interact with the AttendanceEntity for CRUD operations.

3. CompanyStaticsRepository.java:

- Manages company-related static data.
- Helps in retrieving metrics and static data about the company.
- CRUD operations for static company-related information, such as company configurations, settings, and statistics.

4. ContactFormRepository.java:

- Manages the incoming contact form submissions.
- Offers the method to store, retrieve, update, and delete contact form entries.
- Interacts with data submitted via contact forms by users.

5. FeedbackRepository.java

- Manages feedback from the users.
- Stores, retrieves, and updates user feedback including feedback forms, reviews, or surveys.
- Interacts with a database to perform CRUD on records related to feedback

6. RateRepository.java

- Manages rate-related data.
- Interacts with the database to perform CRUD operations with the configurations of rates.
- It allows rates to interact with user-specific periods, such as rates by month/year.

7. TokenRepository.java:

- Token-based operations.
- CRUD operations for tokens (e.g., JWT tokens, session tokens, verification tokens).
- Interacts with the database to store and validate tokens.

8. UserRepository.java:

- User-related data management.
- CRUD operations for user entities, including registration, login, and updates.
- Queries related to user data, such as roles, status, and other user-specific details.

AdminRepository

```
package com.example.newdemo.repository;

import com.example.newdemo.model.AdminEntity;

public interface AdminRepository extends JpaRepository<AdminEntity, Long> {

    Optional<AdminEntity> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

5. EntityModel

An entity is a lightweight persistence domain object. Typically, an entity represents a table in a relational database, and each entity instance corresponds to a row in that table.

The Entities used in the project are:

- AdminEntity
- UserEntity
- TokenEntity
- FeedbackEntity
- ContactEntity
- CompanyStaticsEntity
- RateEntity
- AttendanceEntity

1.AdminEntity

```
1 package com.example.newdemo.model;
2
3 import java.time.LocalDate;
4
5
6
7
8
9
10
11 @Entity
12 @Table(name = "admin", uniqueConstraints = @UniqueConstraint(columnNames = "email"))
13 public class AdminEntity {
14
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     @Column(nullable = false, updatable = false)
18     private Long id;
19
20     @NotBlank(message = "Name cannot be empty")
21     private String name;
22
23     @NotBlank(message = "Email cannot be empty")
24     @Pattern(regexp = "^(?!.*\\.\\.\\.)(?!\\.\\.\\.)[A-Za-z0-9!#$%&'*/+=?^`{|}~-]+(?(?:\\.[A-Za-z0-9!#$%&'*/+=?^`{|}~-]+)*)@(?:([A-Za-z0-9](?:[A-Za-z0-9-]{0,62})?)|[A-Za-z0-9-]{2,63})\\.([A-Za-z]{2,6})$", flags = Pattern.CASE_INSENSITIVE | Pattern.UNICODE_CASE)
25     private String email;
26
27     @NotBlank(message = "Password cannot be empty")
28     private String password;
29     @NotBlank(message = "Role cannot be empty")
30     private String role;
31
32     public String getRole() {
33         return role;
34     }
35
36     public void setRole(String role) {
37         this.role = role;
38     }
39
40     public void setTokens(List<TokenEntity> tokens) {
41         this.tokens = tokens;
42     }
43
44     public LocalDate getCreatedAt() {
45         return createdAt;
46     }
47
48     public void setCreatedAt(LocalDate createdAt) {
49         this.createdAt = createdAt;
50     }
51
52     public LocalDate getUpdatedAt() {
53         return updatedAt;
54     }
55
56     public void setUpdatedAt(LocalDate updatedAt) {
57         this.updatedAt = updatedAt;
58     }
59
60     @PrePersist
61     protected void onCreate() {
62         LocalDate now = LocalDate.now();
63         this.createdAt = now;
64         this.updatedAt = now;
65     }
66
67     @PreUpdate
68     protected void onUpdate() {
69         this.updatedAt = LocalDate.now();
70     }
71 }
```

```
public void setTokens(List<TokenEntity> tokens) {
    this.tokens = tokens;
}
```

```
public LocalDate getCreatedAt() {
    return createdAt;
}
```

```
public void setCreatedAt(LocalDate createdAt) {
    this.createdAt = createdAt;
}
```

```
public LocalDate getUpdatedAt() {
    return updatedAt;
}
```

```
public void setUpdatedAt(LocalDate updatedAt) {
    this.updatedAt = updatedAt;
}
```

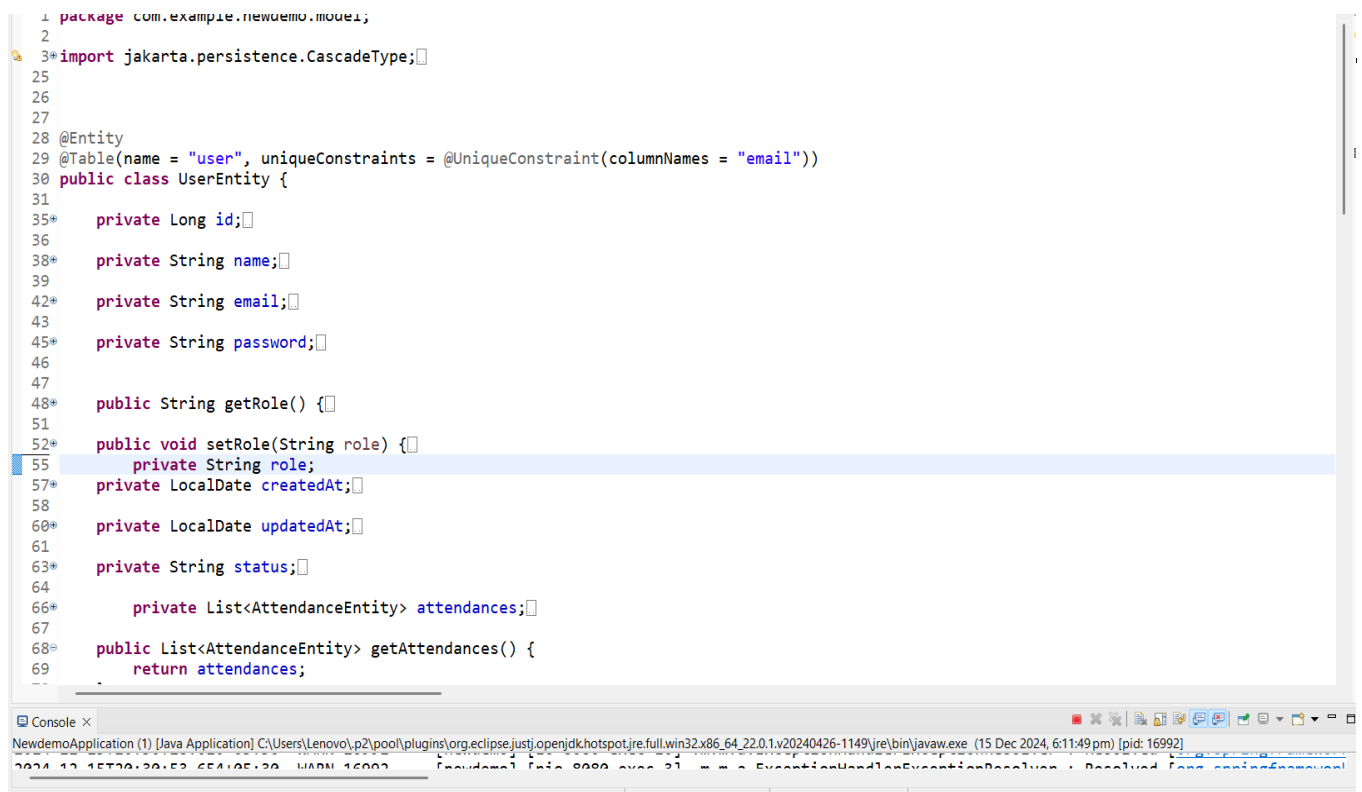
```
@PrePersist
protected void onCreate() {
    LocalDate now = LocalDate.now();
    this.createdAt = now;
    this.updatedAt = now;
}
```

```
@PreUpdate
protected void onUpdate() {
    this.updatedAt = LocalDate.now();
}
```

```
}
```

2.UserEntity

```
1 package com.example.newdemo.model;
2
3 import jakarta.persistence.CascadeType;
4
5
6
7
8 @Entity
9 @Table(name = "user", uniqueConstraints = @UniqueConstraint(columnNames = "email"))
10 public class UserEntity {
11
12     private Long id;
13
14     private String name;
15
16     private String email;
17
18     private String password;
19
20     public String getRole() {}
21
22     public void setRole(String role) {}
23     private String role;
24     private LocalDate createdAt;
25     private LocalDate updatedAt;
26     private String status;
27
28     private List<AttendanceEntity> attendances;
29
30     public List<AttendanceEntity> getAttendances() {
31         return attendances;
32     }
33 }
```



5. Service Layer

In a Spring Boot REST API application, a service layer is a crucial architectural component that plays several important roles: Separation of Concerns: The service layer helps in separating business logic from the concerns of handling HTTP requests and responses (which are typically handled in controllers)

The Services used in the project are:

- AdminService
- UserService
- TokenService
- FeedbackService
- ContactService
- RateService
- AttendanceService

1.AdminService

```
1 package com.example.newdemo.service;
2
3 import com.example.newdemo.exception.ValidationUtil;
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 @Service
27 public class AdminService {
28
29     @Autowired
30     private UserRepository userRepository;
31
32     @Autowired
33     private AdminRepository adminRepository;
34
35     @Autowired
36     private ModelMapper modelMapper;
37
38     public ResponseAdmin insertNewAdmin(AdminEntity newadmin) {
39         validateEmail(newadmin.getEmail());
40         String formattedName = validateAndFormatName(newadmin.getName());
41         newadmin.setName(formattedName);
42         validatePassword(newadmin.getPassword());
43
44         // Check if the admin email already exists
45         if (adminRepository.findByEmail(newadmin.getEmail()).isPresent()) {
46             throw new ValidationUtil.UserAlreadyExistsException("Admin with this email already exists");
47         }
48
49         // Check if the user email already exists
50         if (userRepository.findByEmail(newadmin.getEmail()).isPresent()) {
51             throw new ValidationUtil.UserAlreadyExistsException("User with this email already exists");
52         }
53
54         newadmin.setPassword(PasswordUtility.hashPassword(newadmin.getPassword()));
55         AdminEntity createdAdmin = adminRepository.save(newadmin);
56         ResponseAdmin response = modelMapper.map(createdAdmin, ResponseAdmin.class);
57     }
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
        responseUser.setUpdatedAt(userEntity.getUpdatedAt());
        responseUser.setStatus("Active"); // Default status, modify as needed
        responseUser.setMessage("User data successfully retrieved");
        return responseUser;
    }

    public String deleteUserById(Long id) {
        if (userRepository.existsById(id)) {
            userRepository.deleteById(id);
            return "Deleted user with ID " + id;
        } else {
            throw new ValidationUtil.UserNotFoundException("User not found");
        }
    }

    public String deleteAllUsers() {
        userRepository.deleteAll();
        return "All users are deleted";
    }

    private String formatName(String name) {
        // Split the name into parts and ensure the format 'FirstName LastName'
        String[] parts = name.split("\\s+");
        if (parts.length < 2) {
            return null; // Invalid format
        }
        // Reassemble the name with exactly one space
        return String.join(" ", parts).trim();
    }
}
```

2.UserService

```
1 package com.example.newdemo.service;
2
3 import java.time.LocalDate;
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22 @Service
23 public class UserService {
24
25     @Autowired
26     private UserRepository userRepository;
27     @Autowired
28     private AttendanceRepository attendanceRepository;
29
30     public Map<String, Object> getUsersWithAttendance(LocalDate date) {}
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70     @Autowired
71     private CompanyStaticsRepository companyStaticsRepository;
72
73     public List<CompanyStatics> getAllCompanyStatics() {
74         return companyStaticsRepository.findAll();
75     }
76
77     public AttendanceSummaryDTO getAttendanceSummary(Long userId, int month, int year) {}
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109 }
110
```

CHAPTER-05

TESTING

Testing is a process of executing a program with an intent of finding an error. A good test case is one that has a high probability of finding an as yet undiscovered error. A successful test is one that uncovers a yet undiscovered error. Testing is the major quality control measure used during software development. Its basic function is to detect errors in the software. Testing not only uncovers errors introduced during coding, but also errors introduced during the previous phases.

Thus, the goal of testing is to uncover requirement, design and coding errors in the programs.

The starting point of testing is unit testing. In this, a module is tested separately and is often performed by the coder himself simultaneously along with the coding of the module.

The purpose is to exercise the different parts of the module code to detect coding errors.

After this, modules are integrated into subsystems and subsystems are integrated eventually forming the whole system. Module integration involves integration testing to pick design errors by focusing on the interconnection between modules. Once the system is put together, system testing is conducted. Here the system is tested against the system requirements to see if all requirements are met and if the system performs as Specified by the requirements. Lastly, acceptance testing is carried out to demonstrate to the client, on the real life data of the client, the operation of the system.

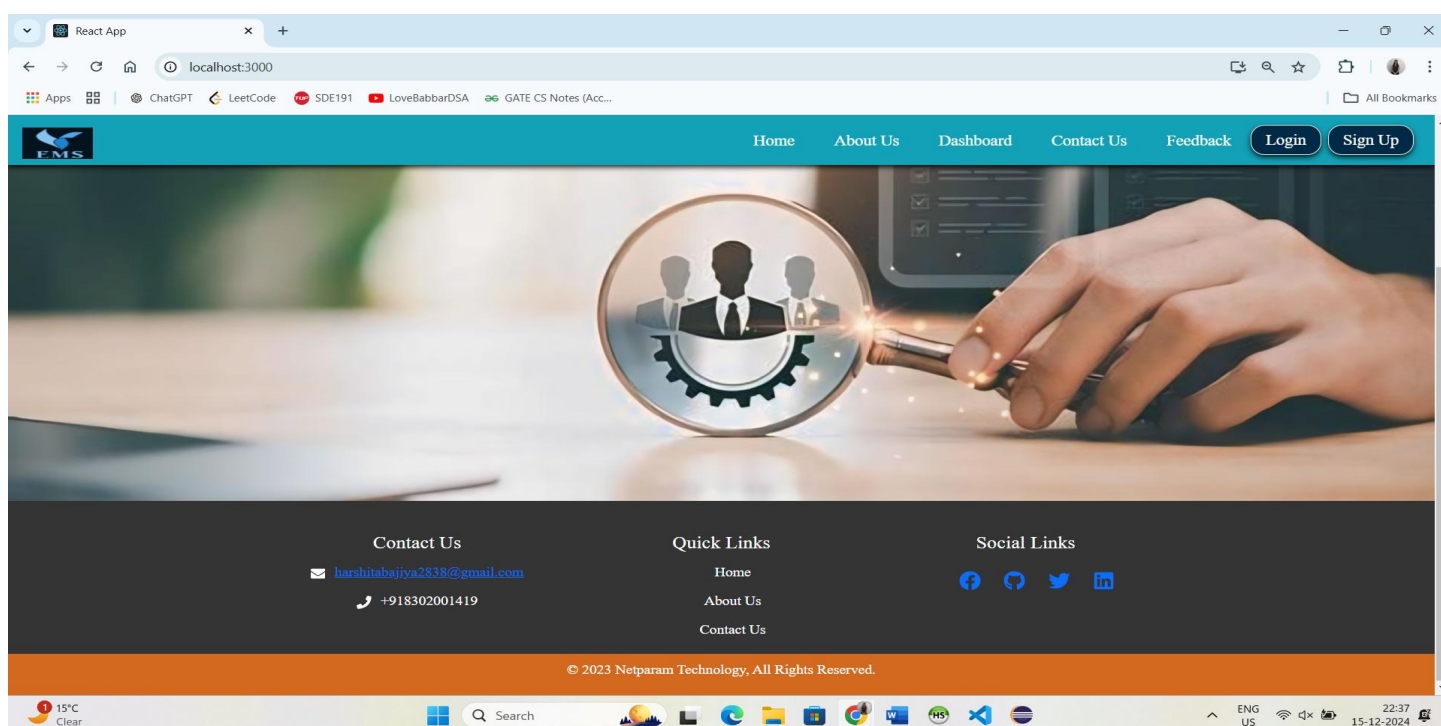
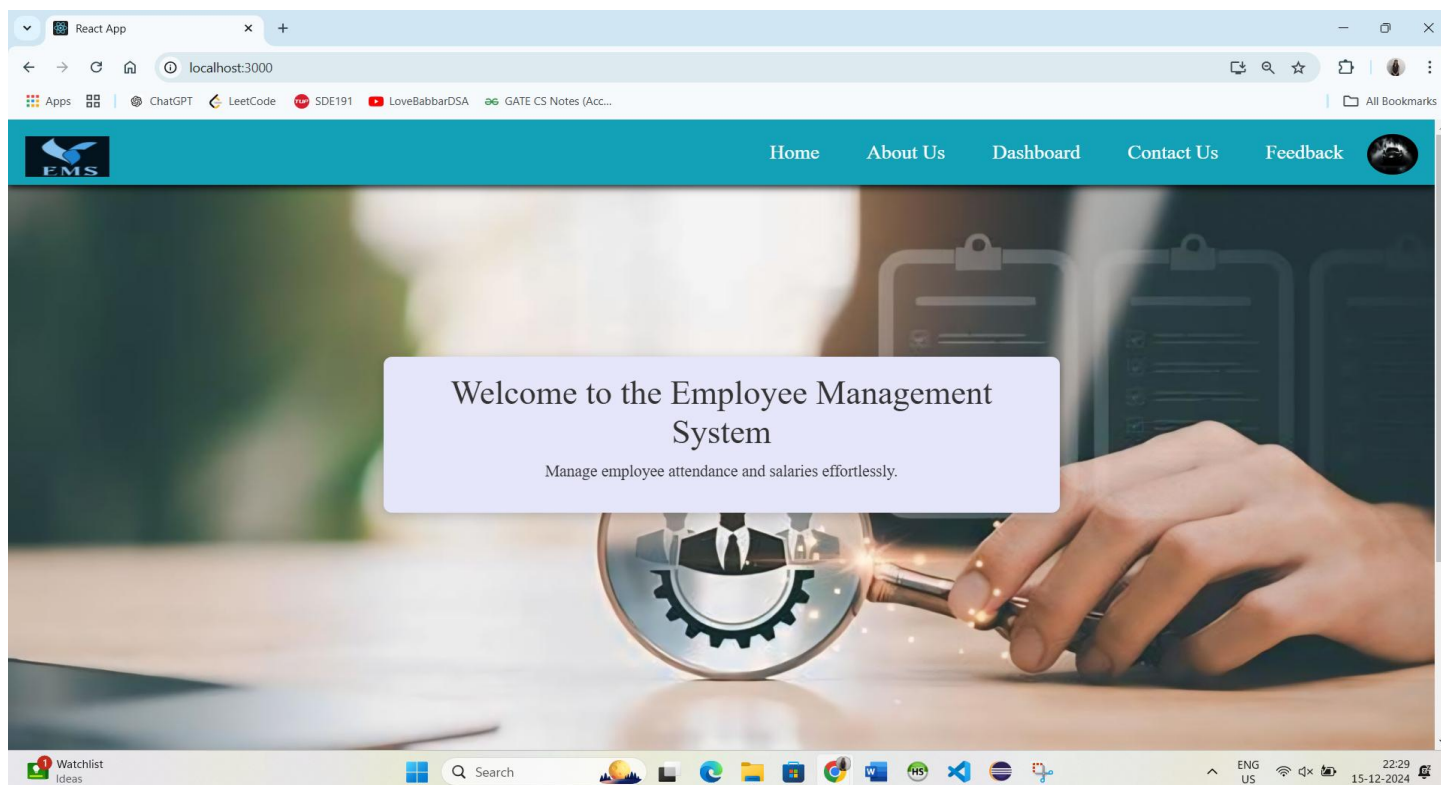
Depending upon the intensity and formality of testing, it can be of two types. CMS undergoes both types of testing.

CHAPTER-06

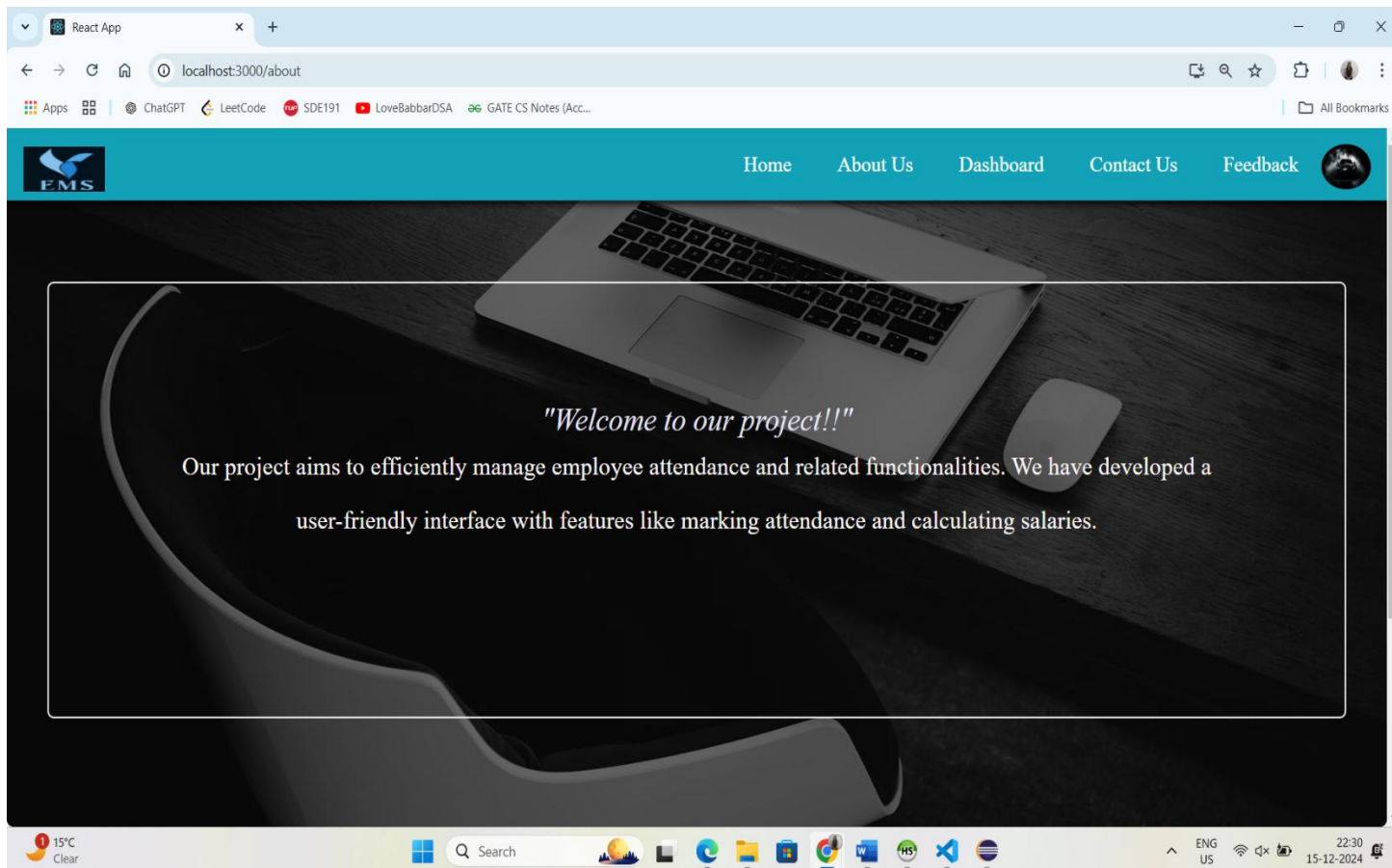
USER INTERFACES

6.1 Home page

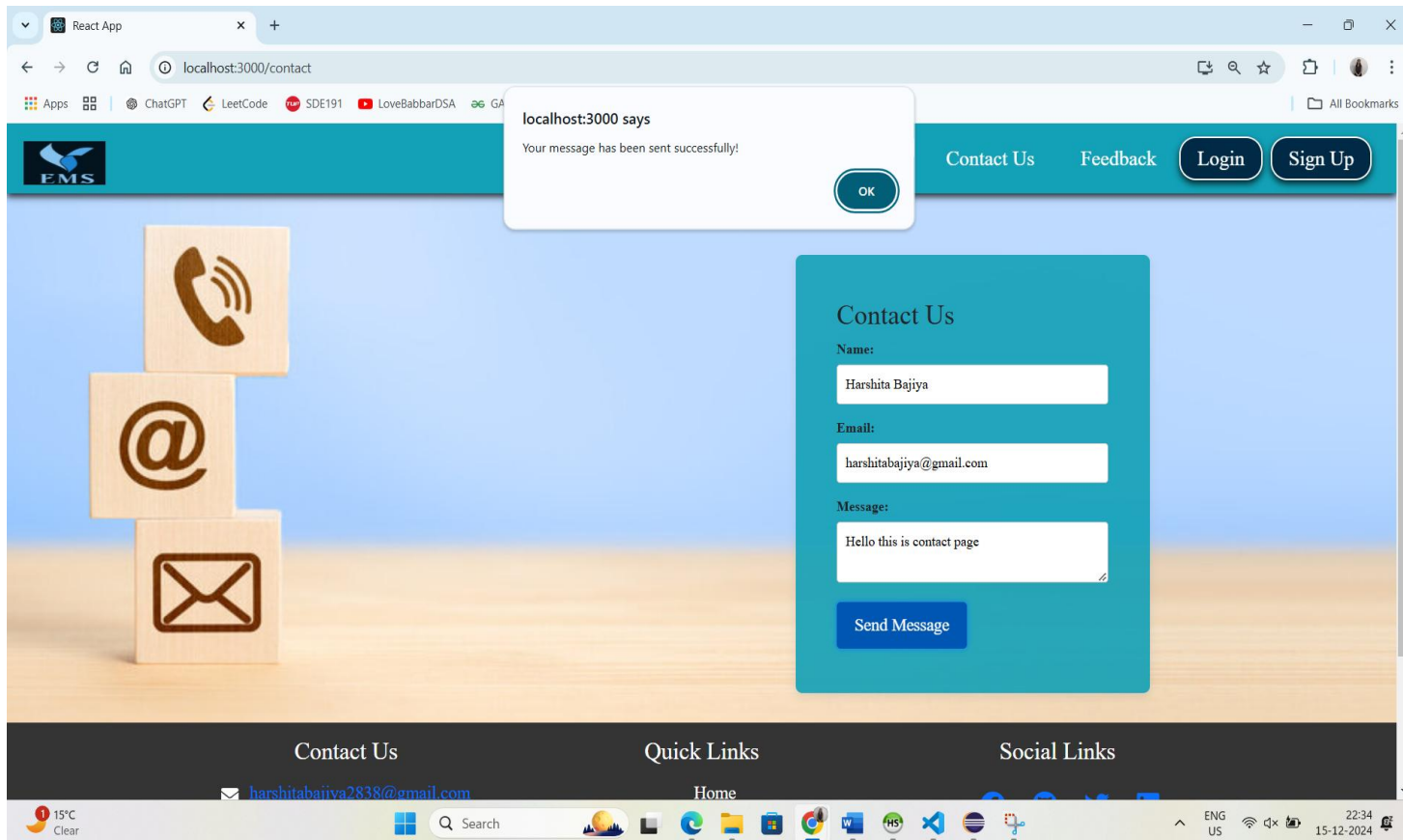
This is welcome page for the user , The UIs are designed to facilitate easy navigation, efficient data retrieval, and seamless user experience.



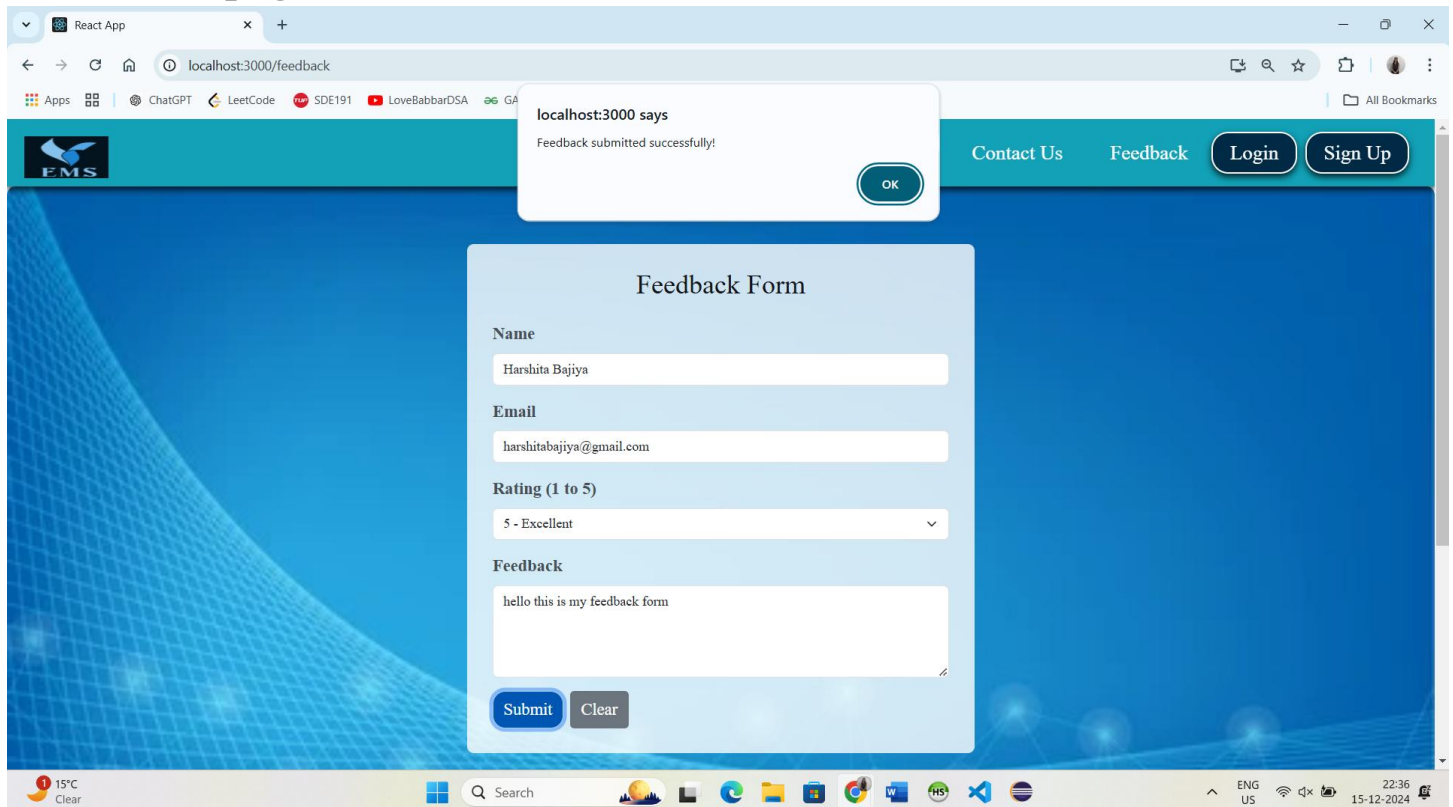
2.AboutUs-page



3.Contact-Us page



4.Feedback page



5.Login Page

The screenshot shows a web browser window with the URL `localhost:3000/login`. The page features a teal header with the EMS logo and navigation links: Home, About Us, Dashboard, Contact Us, Feedback, Login, and Sign Up. The main content area has a blue background with a network pattern. A white login form is centered, containing fields for Email and Password, a green Login button, and a blue Sign Up button. A link "Don't have an account?" is positioned between the Login and Sign Up buttons.

EMS Home About Us Dashboard Contact Us Feedback Login Sign Up

Login

Email:

Password:

Login

Don't have an account?

Sign Up

6.SignUp page

The screenshot shows a web browser window with the URL `localhost:3000/signup`. The page features a teal header with the EMS logo and navigation links: Home, About Us, Dashboard, Contact Us, Feedback, Login, and Sign Up. The main content area has a blue background with a network pattern. A white sign-up form is centered, containing fields for Name, Email, Password, and a Role dropdown menu. A blue Sign Up button is at the bottom, with a link "Already have an account? Sign In" below it.

EMS Home About Us Dashboard Contact Us Feedback Login Sign Up

Sign Up

Name:

Email:

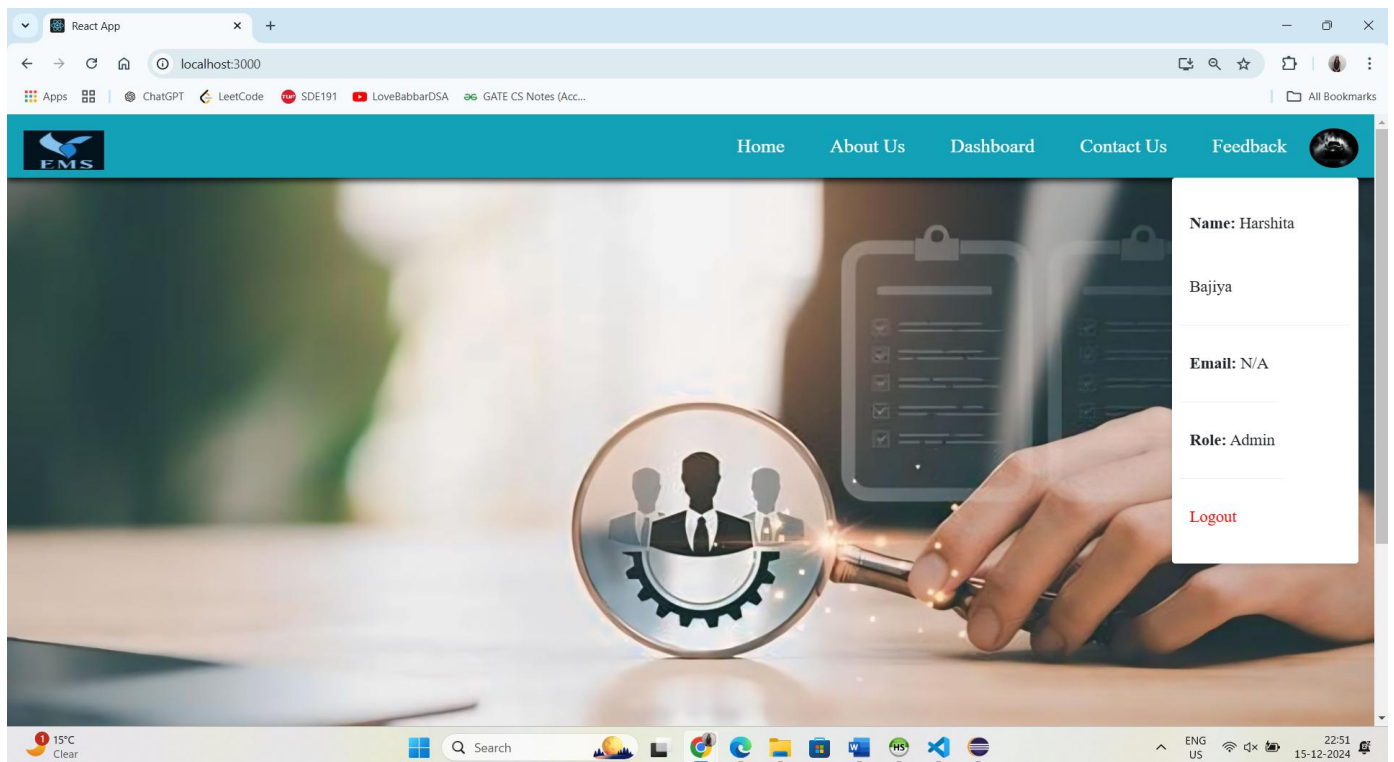
Password:
 Show

Role:
Select Role

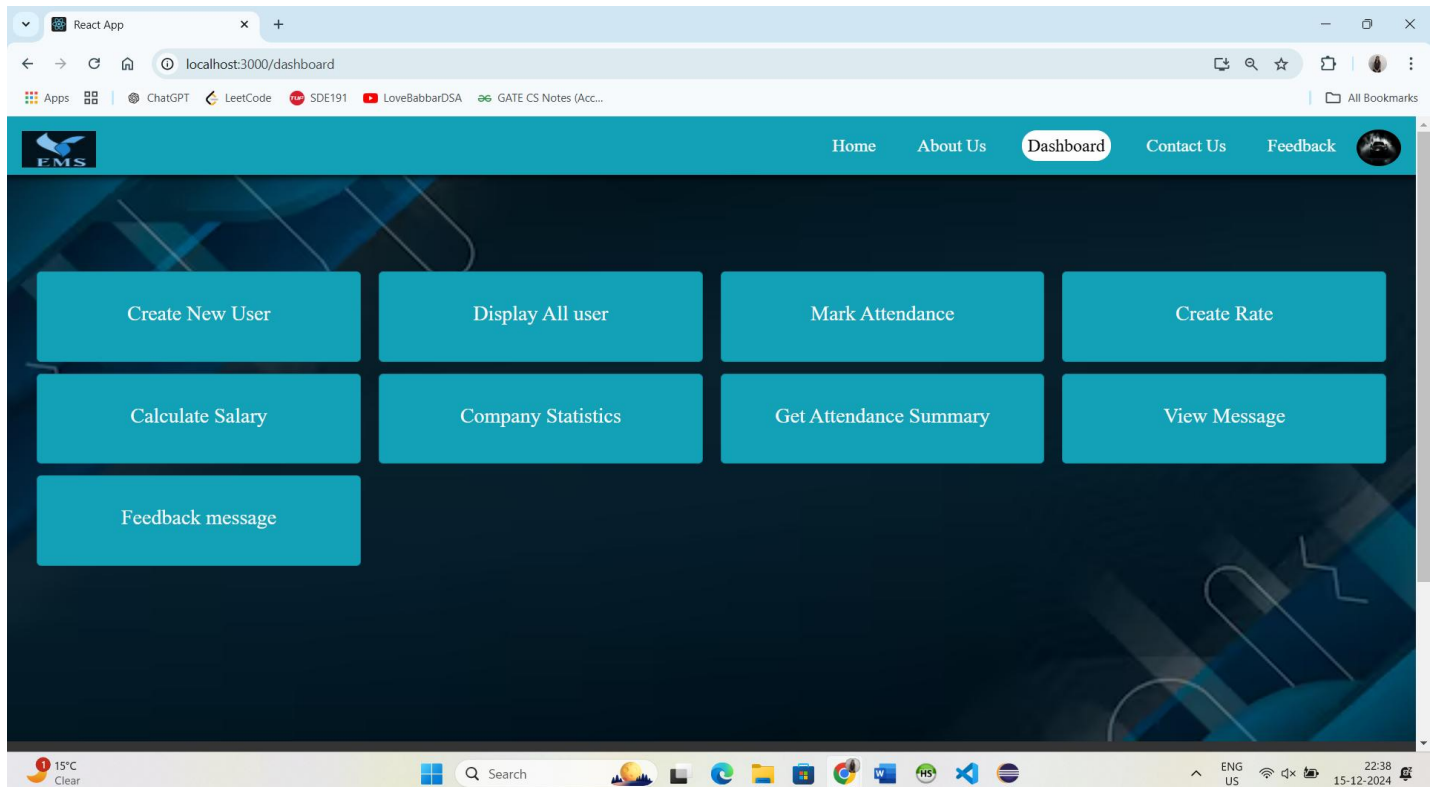
Sign Up

Already have an account? [Sign In](#)

7.UserProfile page



8.DashBoard page



9.Create new User Page

The screenshot shows a web browser window with the URL `localhost:3000/createEmp`. The page features a teal header with the 'EMS' logo and navigation links: Home, About Us, Dashboard, Contact Us, and Feedback. The main content area has a blue background with a white modal form titled 'Create new Employee'. The form contains the following fields:

- Name:** A text input field containing 'Ram Singh Chauhan'.
- Email:** A text input field containing 'ram@gmail.com'.
- Password:** A text input field containing 'Ram@123', with a toggle icon for visibility.
- Role:** A dropdown menu with 'Full Stack Developer' selected.
- Buttons:** A blue 'Submit' button and a grey 'Clear' button.

The Windows taskbar at the bottom shows the system clock as 22:40 on 15-12-2024.

10.Display All users page

The screenshot shows a web browser window with the URL `localhost:3000/display`. The page features a teal header with the 'EMS' logo and navigation links: Home, About Us, Dashboard, Contact Us, and Feedback. The main content area has a blue background with a white modal form titled 'Display Details'.

ID	Email	Name	Role	Status	Actions
1	harshita123@gmail.com	Harshita Bajiya	user	Active	<button>View</button> <button>Edit</button> <button>Delete</button>
6	kajal@gmail.com	Kajal Kumari	Backend Developer	Active	<button>View</button> <button>Edit</button> <button>Delete</button>
7	rain@gmail.com	Ruchi Jain	Frontend Developer	Active	<button>View</button> <button>Edit</button> <button>Delete</button>
10	de@gmail.com	abcd deff	Frontend Developer	Active	<button>View</button> <button>Edit</button> <button>Delete</button>

On the right side of the page, there is a sidebar for user details for 'Kajal Kumari' (ID: 6). The sidebar includes a 'Close' button, the user's ID, email, and role. Below this, there is a 'Monthly Rates' section with a table showing rates for November and December.

Month	Rate
NOVEMBER	900
DECEMBER	1000

The Windows taskbar at the bottom shows the system clock as 22:44 on 15-12-2024.

11.Mark Attendance page

React App

localhost:3000/markattendance

Attendance marked successfully!

About Us Dashboard Contact Us Feedback

EMS

Attendance Table

Select Date:

2024-12-15

Sequence	User ID	Name	Role	Attendance Type	Hours	Actions	Status
1	1	Harshita Bajiya	user	Select Attendance Type	9	Submit	Marked
2	6	Kajal Kumari	Backend Developer	Half Day	Enter hours	Submit	Marked
3	7	Ruchi Jain	Frontend Developer	Select Attendance Type	5	Submit	Marked
4	10	abcd deff	Frontend Developer	Select Attendance Type	Enter hours	Submit	Not Marked

15°C Clear

Search

ENG US

22:46

15-12-2024

12.Create Rate

React App

localhost:3000/createrate

Rate Management

Rate:

1600

Effective From:

01-12-2024

Effective Till:

31-12-2024

User ID:

1

Submit

Home About Us Dashboard Contact Us Feedback

EMS

15°C Clear

Search

ENG US

22:47

15-12-2024

13. Calculate Salary

React App

localhost:3000/calculateTotalRating

Apps ChatGPT LeetCode SDE191 LoveBabbarDSA GATE CS Notes (Acc... All Bookmarks

EMS Home About Us Dashboard Contact Us Feedback

Calculate Total Salary/Month

User ID:

Month:

Year:

Calculate Salary

Total Salary: 49500

14. Company Statics

React App

localhost:3000/companystatics

Apps ChatGPT LeetCode SDE191 LoveBabbarDSA GATE CS Notes (Acc... All Bookmarks

EMS Home About Us Dashboard Contact Us Feedback

Company Statics

Company Name	Full Day Hours	Half Day Hours
NetParam Technology	9	5
Banasthali_Vidyapeeth	9	5

Contact Us

✉ harshitabajjya2838@gmail.com

☎ +918302001419

Quick Links

Home

About Us

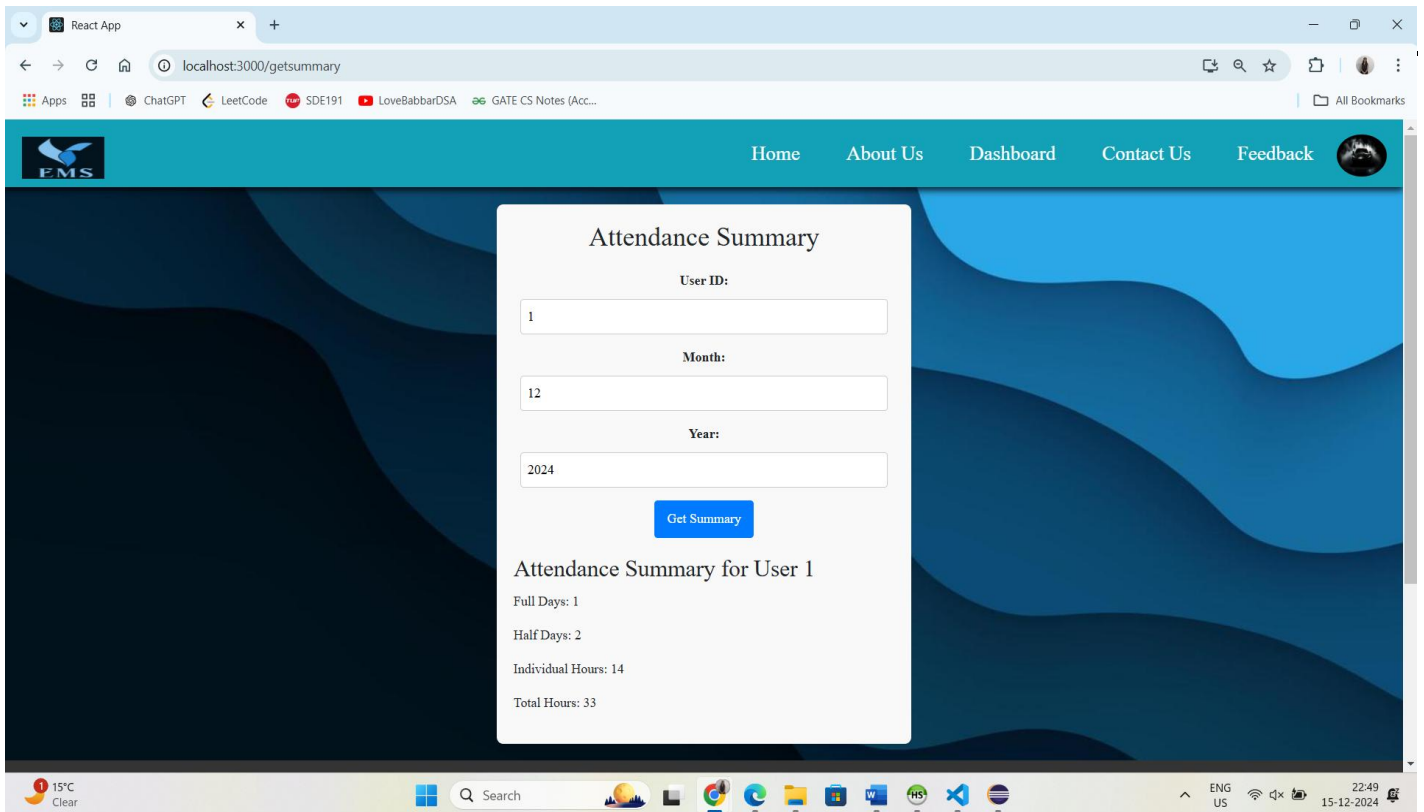
Contact Us

Social Links

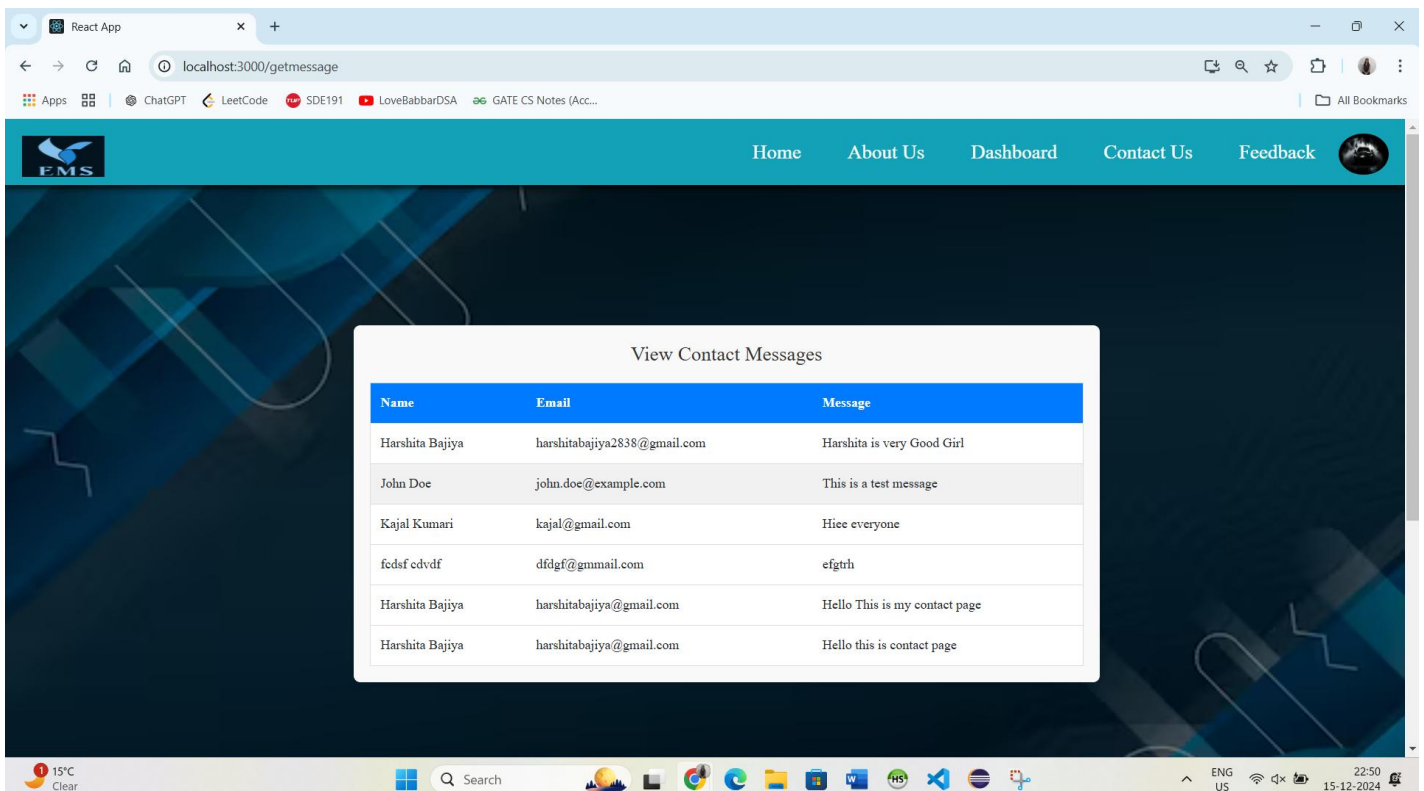
[Facebook](#) [Twitter](#) [LinkedIn](#)

15°C Clear Search 22:48 15-12-2024

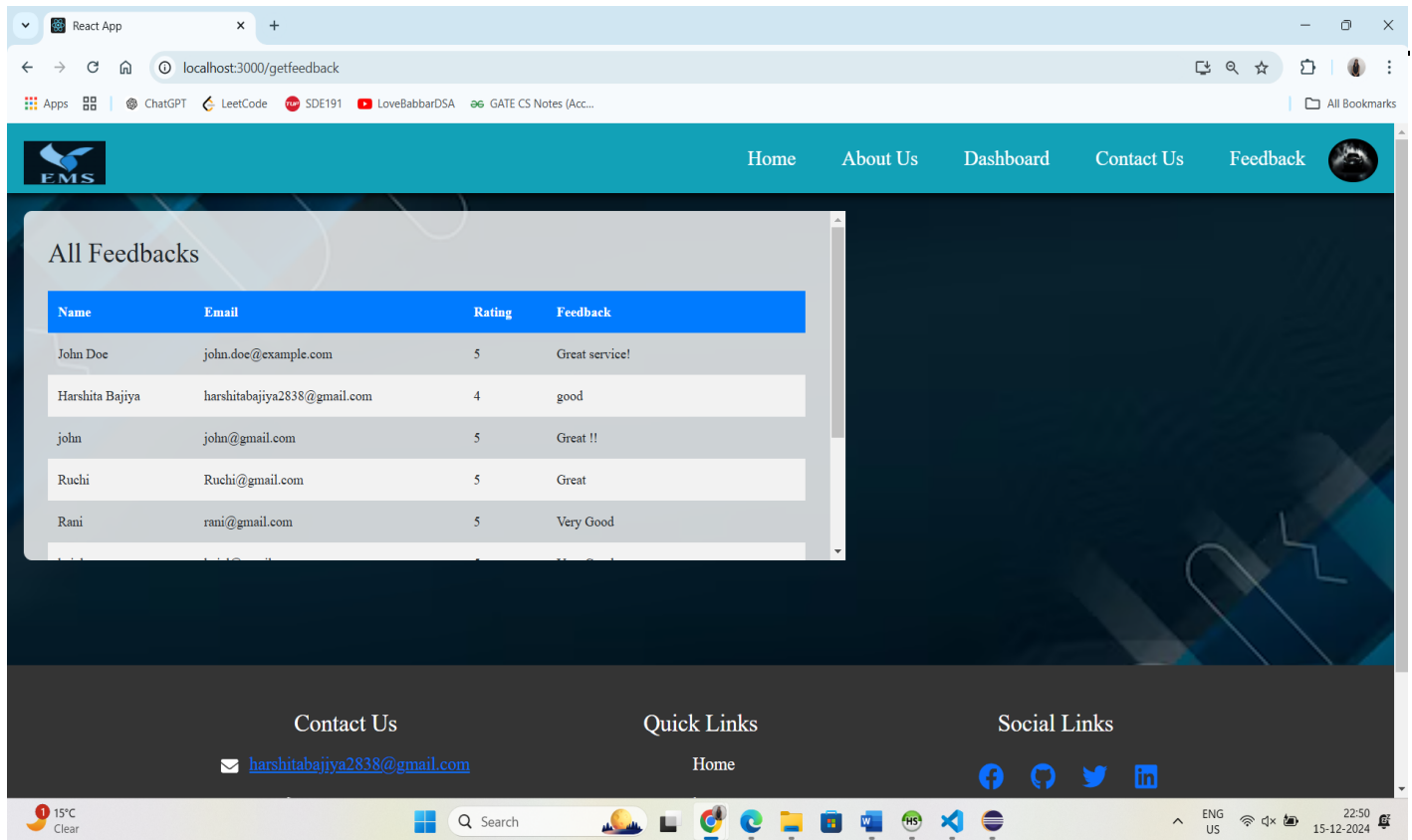
15. Get Attendance Summary page



16. View Message sent by users page



17.View Feedback Message



The screenshot shows a web browser window with the address bar displaying 'localhost:3000/getfeedback'. The browser's bookmark bar includes links to 'Apps', 'ChatGPT', 'LeetCode', 'SDE191', 'LoveBabbarDSA', and 'GATE CS Notes (Acc...'. The web application has a teal header with the 'EMS' logo on the left and navigation links for 'Home', 'About Us', 'Dashboard', 'Contact Us', and 'Feedback' on the right. A user profile picture is also visible in the top right corner.

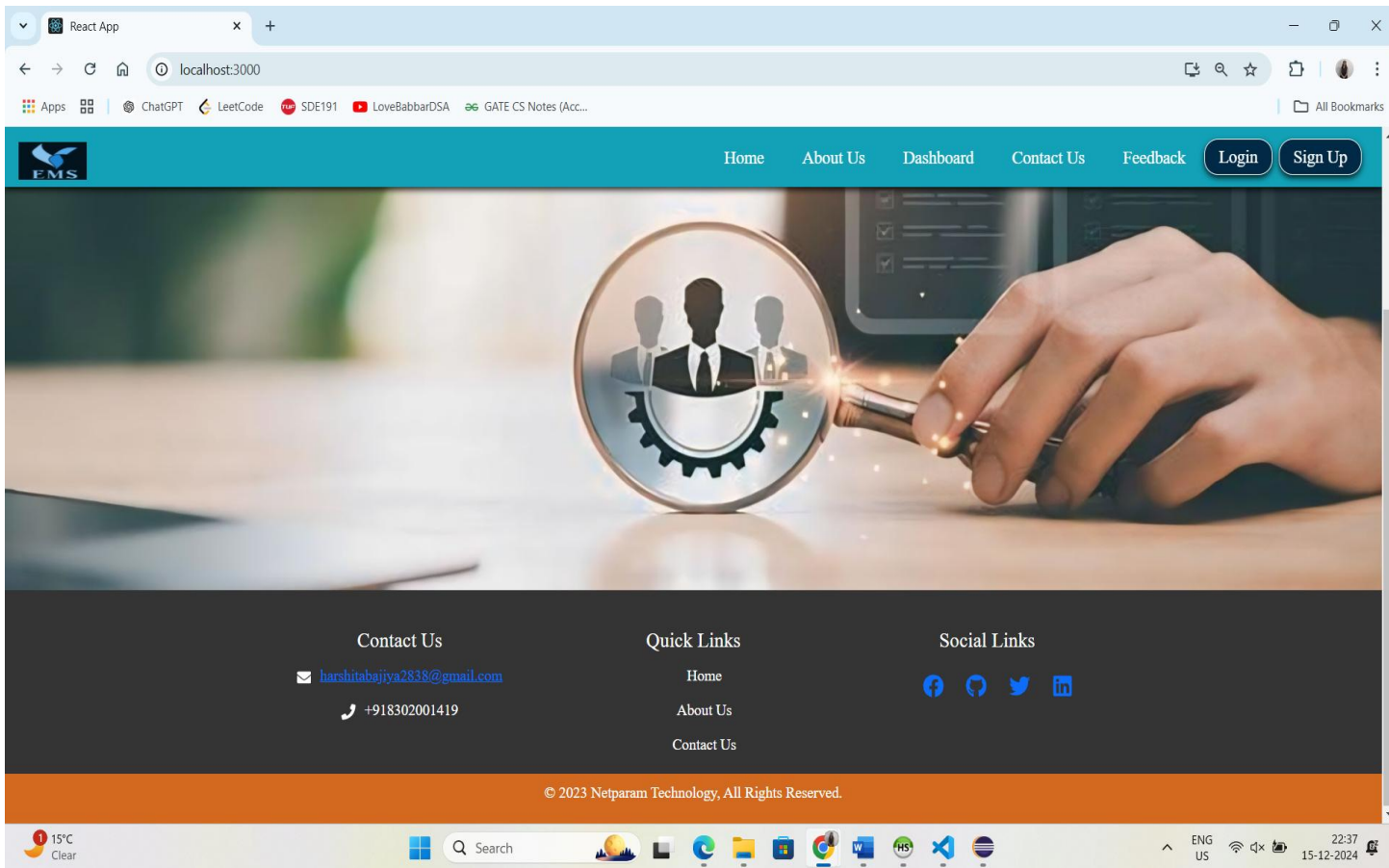
The main content area features a section titled 'All Feedbacks' which contains a table with the following data:

Name	Email	Rating	Feedback
John Doe	john.doe@example.com	5	Great service!
Harshita Bajiya	harshitabajiya2838@gmail.com	4	good
john	john@gmail.com	5	Great !!
Ruchi	Ruchi@gmail.com	5	Great
Rani	rani@gmail.com	5	Very Good

The footer of the application is dark grey and divided into three sections: 'Contact Us' with an email icon and the address 'harshitabajiya2838@gmail.com'; 'Quick Links' with a 'Home' link; and 'Social Links' with icons for Facebook, GitHub, Twitter, and LinkedIn.

The Windows taskbar at the bottom shows the system clock at 22:50 on 15-12-2024, the language set to ENG US, and various system icons including network, volume, and battery.

Employee Management System



CONCLUSION

I have successfully designed and developed a webpage, which is demonstrated in a way so that all the employee details are stored securely, employee can send the leave easily and can generate the pay slip. The system was implemented using web-based technologies which include CSS, JS, Bootstrap, HTML, React, MySQL, and Spring Boot. The system serves as improvement in employee management, maintain accuracy, transparency and highlight the need to integrate advance technology in employee record. In future I will add on some extra feature and functionality that the developed system enables the employees to request and track their leave at their own convenient time in timely manner. Superior officers of the institution and administrative department can create leave policies, check transparency and plan activities ahead of time. Employee Leave Management System will provide an effective way of managing the different employee leave requests. The system will eliminate all the problems encountered in the manual method which will help the companies or organization better improve their business operations

Appendices

✓ **Tools and Technologies Used**

- **Frontend:** React.js, HTML, CSS, Bootstrap
- **Backend:** Spring Boot (Java).
- **Database:** MySQL (HeidiSQL).
- **API Testing:** Postman.

✓ **Code Snippets**

✓ **Data Flow Diagrams (DFD)**

✓ **Use Cases**

✓ **Activity Diagram**

✓ **ER Diagram**

✓ **Sequence Diagram**

✓ **Flowcharts**

✓ **Screenshots of the System**

- Login Page.
- Admin Dashboard.
- Employee Attendance Management
- Attendance Marking
- Display all Users
- Company Statics etc

✓ **Future Enhancements**

✓ **Glossary**

- EMS: Employee Management System.
- DFD :Data Flow Diagram
- ERD : Entity-Relationship Diagram
- API: Application Programming Interface.

REFERENCES

- <https://spring.io/projects/spring-boot>
- <https://start.spring.io/>
- <https://www.heidisql.com/download.php>
- <https://www.eclipse.org/downloads/>
- <https://www.mysql.com/downloads/>
- <https://www.javatpoint.com/spring-boot-tutorial>
- <https://www.javatpoint.com/spring-boot-crud-operations>
- <https://www.geeksforgeeks.org/spring-boot-map-entity-to-dto-using-modelmapper/>
- <https://www.geeksforgeeks.org/spring-boot-map-entity-to-dto-using-modelmapper/>
- <https://www.javatpoint.com/mysql-tutorial>